

HarnessDev: 大语言模型能否创建并演化自身的智能体执行框架?

¹ByteDance Seed, ²Singapore University of Technology and Design,
³Georgia Institute of Technology, ⁴M-A-P, ⁵TokenWave.AI

完整作者列表见贡献说明

Abstract

随着智能体从研究原型走向部署工具，其能力日益依赖于模型外部的执行基础设施，通常称为智能体执行框架（Agent Harness）。在固定模型权重的情况下，改变该执行框架可显著改变任务性能。当前的智能体评估通常在选定执行框架下报告下游性能，而对模型自身开发执行框架的能力关注相对不足。本文提出 HarnessDev 基准，将评估单元从任务输出转向可运行的基础设施。HarnessDev 涵盖两个阶段。在创建阶段，智能体从最小种子和少量案例出发，构建完整的执行系统。在演化阶段，智能体从自身创建的执行框架出发，利用下游执行反馈进行迭代修订，目标是提升基准性能。随后，本文从能力——在留出基准上的任务成功率，以及效率——执行令牌成本两方面评估每个构建的执行框架。所报告的创建结果涵盖六种创建者大语言模型、四个领域和五个下游基准，共计 2,207 个独立下游实例，其中隐藏评估任务在开发阶段不可见。研究发现，生成的执行框架在代码、搜索和研究任务上仍显著落后于成熟的人工工程参考框架，而在写作和机器学习实验任务上达到或超过所选参考框架，且执行成本差异较大。演化带来一定的性能提升，但提升不稳定，且仅部分迁移至留出任务。使用固定运行时模型的实验进一步表明，这些提升强烈依赖于执行该框架的模型，表明跨模型迁移有限。

Date: September 2, 2026

Project Page: <https://self-developing-agents.github.io/>

1 引言

随着智能体从研究原型走向部署工具，如编码助手 [2, 32]、数据分析副驾驶、浏览器工作者 [5] 和研究流水线，其能力越来越依赖于模型权重之外的软件。这一外围执行基础设施通常被称为智能体框架（Agent Harness）[31, 39]，它管理执行循环、工具使用、上下文、故障恢复和结果验证，将模型输出转化为行动 [3]。其影响是显著的：在相同权重下，GPT-5 在 Terminus 2 中解决 Terminal-Bench 2.1 的比例为 35.2%，而在 Codex CLI 中为 49.6% [45]。随着智能体向更多领域专业化，对专用框架的需求将持续增长。由于这些系统需要持续开发而非一次性实现，一个实际的问题是，大语言模型（LLM）能否协助框架工程师——甚至接管这一角色——构建并持续改进框架。

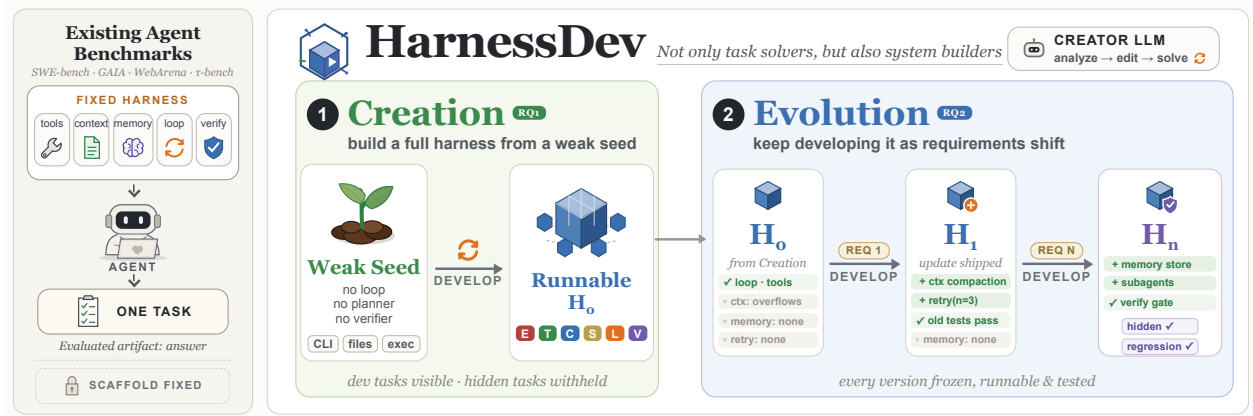


图 1 HarnessDev 涵盖框架开发的两个阶段。在创建阶段，创建者从一个弱但可运行的种子和少量开发案例构建完整的框架。在演化阶段，它利用下游执行反馈持续改进自身的持久框架。两个阶段都评估跨任务持久存在的可运行基础设施，而非一次性任务输出。

尽管存在这一实际需求，大多数智能体评估为给定比较选择一个框架，并报告模型在下游任务上的性能 [18, 26, 29, 54, 61]。这种设置支持受控的任务级比较，但将框架视为实验配置的一部分，而非待开发的工件。近期工作已开始研究框架表示、自动化智能体设计以及构建或改进智能体系统的智能体 [15, 27, 31, 39, 56, 57]。然而，模型能否既创建又持续改进可运行的持久框架，仍未得到充分探索。回答这一问题需要将开发框架的模型与执行下游任务的模型分离，记录开发环境，并测量下游性能、跨执行器的迁移、与人工工程系统的距离、回归和成本。

这种评估差距尤为关键，因为测试框架工程与普通代码编辑有着根本性的不同。当模型修改一个独立程序时，目标行为由外部指定，成功与否可以在局部验证。当模型修改自身的测试框架时，它是在编辑自身赖以行动的底层执行环境：这一改变会影响模型在未来所有任务中观察、规划和恢复的方式。因此，有效的测试框架改进要求模型能够从执行轨迹 [42] 中识别自身的行为局限，诊断其运行所在系统的结构性瓶颈，并提交有针对性的修改，这些修改能够累积为持久、可复用的能力提升，而非一次性的修补 [49, 53]。随着前沿模型的能力足以编辑多文件代码库并完成真实的拉取请求，这种能力已经潜在地存在；所缺少的只是一个能够衡量它的基准。

我们提出了 **HarnessDev**（图 1），这是一个填补这一空白的基准，它将评估单元从任务输出转移到可运行的基础设施上：衡量模型构建和维护持久、可检查、可复用的执行系统的能力。该名称反映了软件工程中“开发”的含义：开发一个测试框架既包括从零开始构建（即创建阶段），也包括通过持续迭代和维护进行改进（即演化阶段）。该基准涵盖了测试框架开发的两个阶段。在创建阶段，一个创建者大语言模型从一个故意设计得较弱但可运行的种子开始，为一个新的任务族构建完整的测试框架。在演化阶段，它从一个现有的测试框架开始，继续开发以提升下游任务性能。这两个阶段共同评估模型是否能够完成测试框架开发任务并持续改进所得到的系统。

评估一个生成的测试框架比评估一个生成的答案更难。一个测试框架可能会过拟合于编写它的模型，记忆开发示例，在提升一种能力的同时悄然导致另一种能力退化，或者通过针对基准的特定修改来提高反馈集得分，而这些修改无法迁移到新任务上。因此，我们从两个维度进行评估。能力衡量测试框架是否有效：我们在留出的下游任务上运行它，并报告任务级别的成功率。效率衡量在部署解决下游任务时，冻结的测试框架所消耗的处理器模型令牌数量。

我们的发现遵循工具开发的两个阶段：

工具创建。当前模型能够从弱种子构建可运行的工具，但与成熟人工工程系统的差距因工具类型而异。当每个工具由构建它的模型运行时，模型构建的工具在短篇写作上与参考工具相当，在机器学习实验上则超过参考工具。差距最大的是搜索和研究工具，它们需要长周期的信息检索；代码工具的差距仍然显著，因为代码工具必须在多轮交互中协调仓库检查、编辑和验证。不同创建者模型构建的工具在下游任务性能上差异显著，在消耗的执行器令牌数量上也是如此。更高的执行成本并不一定带来更好的结果，因此工具质量必须同时从能力和效率两方面评估。

工具演化。当前模型能够利用下游执行反馈改进自己的工具，但可靠的演化仍然困难。性能在连续修订中经常起伏，开发过程中观察到的增益在未见任务上变得更小且更不一致。结果还强烈依赖于运行工具的模式：更换运行时模型会改变起始性能以及后续修订是否有帮助。这些结果表明，模型能够做出有用的局部改进，但在未见任务和运行时模式上的稳健演化仍然是一个开放的挑战。

2 背景

大多数智能体基准（agent benchmark）始于问题已经可执行之后：任务已明确，奖励或评判器已定义，执行脚手架已固定。这种设置对于受控比较是必要的，但它掩盖了在实际部署中占主导地位的工作。在工业界，这些工作分散在多个角色中——解决方案架构师、应用工程师和平台工程师——但其最近最明显的结晶是前向部署工程师（Forward-Deployed Engineer, FDE），这一头衔由帕兰提尔（Palantir）推广，并已被前沿模型公司采用 [36, 38]。FDE 在系统被采用后嵌入客户现场，将通用模型转化为能够针对该客户的数据格式、工作流程和合规约束运行的系统——例如，由于日志可能只能保留固定期限而重写摄取路径，或在跨司法管辖区的合同管道中定位故障。该角色的成功标准不是演示或基准分数，而是部署的系统是否真正被使用、持续运行并不断改进；其失败则作为修复和功能请求反馈到产品中 [36]。前沿模型和数据平台公司中 FDE 招聘的快速增长 [44] 反映了一个简单的事实：一个有能力的模型还不是一个可工作的系统 [30]，而目前这一差距由人类工程师来弥合。

从模型的角度来看，FDE 工作提供了基准设计者通常预设的三类结构。首先，目标是模糊的：非正式的业务意图必须转化为具体的目标、约束和成功标准。其次，反馈信号缺失或不可靠：必须构建测试、评判器、迹（trace）或其他自我评估，才能判断系统是否在改进——“合规”只有在有人编码了此处合规的含义之后才变得可检查。第三，执行系统并不以可用的形式存在：未来任务运行所需的工具、上下文管理、状态、生命周期逻辑和验证接口必须被构建、适配并持续维护——FDE 会随着需求变化而留在系统中，而不是交付一次就离开。

本文聚焦于第三层。在典型的受控智能体评估中，研究人员选定一种智能体配置，并报告该配置下的任务完成情况。因此，测试框架通常是评估设置的一部分，而非被开发的对象。HarnessDev 则探究语言模型能否从薄弱的起点构建这一执行脚手架，并利用反馈加以改进，同时保持约束合规性与留出性能。Aspire 研究广泛的部署需求如何转化为能力增长，S₃ Gym 研究交互经验能否被评判与复用，而 HarnessDev 则聚焦于模型如何构建并维护承载它们的系统。

3 基准

3.1 概述

HarnessDev 评估的是模型所开发的执行系统，而非其为单一任务生成的答案。提交的工件是一个可运行的测试框架，该框架被冻结后在下游任务中重复使用。创建者大语言模型 L_C 在开发环境 D 中工作，以生成可运行的测试框架 H 。开发信号因设置而异，定义见表 1。开发完成后， H 被冻结。随后，执行者大语言模型 L_E 在该框架内运行于下游任务 x ，评估器 J 对输出结果 y 进行评分：

$$(L_C, D) \rightarrow H, \quad (H, L_E, x) \rightarrow y \xrightarrow{J} \text{score}. \quad (1)$$

因此， D 用于构建 H ，而 L_E 仅在 H 被冻结后使用。在实现层面， H 包含执行循环、工具、上下文管理、持久状态、生命周期控制与验证；我们以文字描述这些组件，而非为每个组件再分配符号。

该基准研究测试框架开发的两个阶段。

研究问题 1——创建。模型能否从薄弱但可运行的种子构建有效的测试框架？创建者必须将任务规范与少量开发案例转化为能够泛化到未见任务的基础设施。

研究问题 2——演化。模型能否在改进现有测试框架的同时，保留已正常工作的行为？创建者根据下游执行反馈演化其自身的创建测试框架。我们进一步分析生成的工件与轨迹，包括编辑统计、反馈响应、留出泛化以及跨执行器的迁移。

3.2 开发设置

所有设置均提供可变的开发工作区，但它们在起始框架和创建者可用的信号方面有所不同。表 1 给出了核心区别。

Setting	Starts from	Development signal	Output
Creation	Weak seed H_{seed}	Specification and 1–3 development cases	Final harness H
Evolution	Creator’s RQ1 H_0	Results from a designated feedback set	Frozen paired candidates and a creator-declared final harness

表 1 框架开发设置。 创建构建新框架；进化通过下游执行反馈改进该框架。

弱种子 H_{seed} 。创建应衡量模型设计执行系统的能力，而非复现基准样板代码的能力。因此，每位创建者都会收到相同的 H_{seed} ：一个可运行的兼容层，而非任务求解代理。它解析任务和模型配置，暴露允许的低级工具，并写入所需的结果、轨迹、日志和任务工件。其工具是被动的，仅在框架调用时才会执行操作。

该种子没有代理循环、任务分解、工具策略、上下文管理、持久任务状态、验证器、重试或恢复逻辑，也没有停止规则。它可以发出一次连接探测，但不会尝试执行任务。未经修改时，它会产生空或不完整的工件，并在每个下游基准上得分为零。因此，任何非零的创建分数都必须来自创建者添加的执行逻辑。这一边界很重要，因为真实脚手架组合了许多控制原语，其组合会影响任务性能 [31, 39, 40]。

这种设计避免了两个极端。空仓库会将框架设计与命令行和文件格式设置混在一起；成熟的代理则会泄露正在测试的规划和验证结构。 H_{seed} 消除了设置负担，而不提供解决方案策略。图 2 展示了种子及其开发过程。

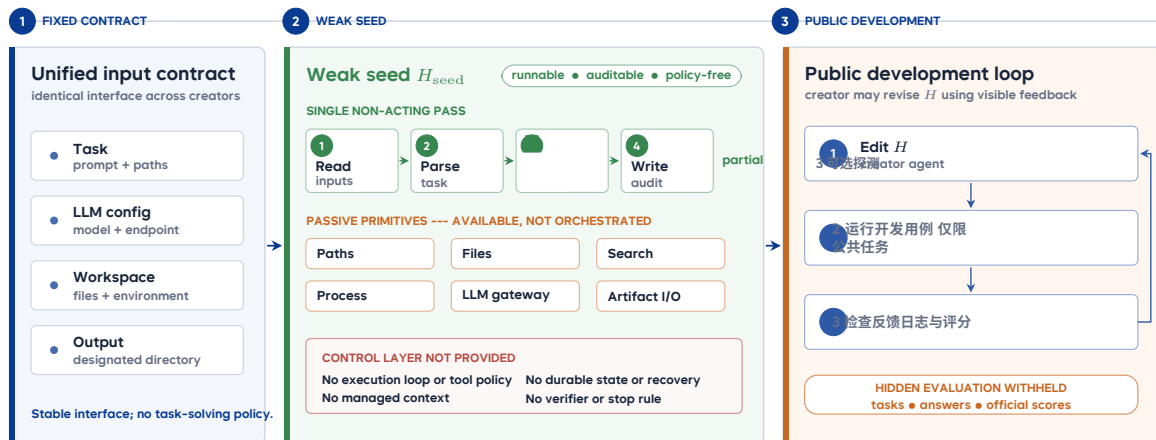


图 2 弱种子测试框架及其开发环境。该种子仅修复输入与审计封装，并暴露被动的、未编排的原语；未经修改时，它不执行任何任务工作。开发反馈是公开的；隐藏任务、答案和官方评分仍不可访问。

环境；图 3 展示了创建者必须实现的控制层，以及完成的测试框架所交付的、评分器可读的工作。附录 C.1 给出了其实现骨架。

创建（研究问题 1）。除了 H_{seed} 之外，创建者还会收到任务族规范、工具与权限约束、简短的设计教程，以及一到三个开发案例。它可以根据这些案例的反馈修改测试框架，但永远不会看到人类实现或隐藏评估集。由此产生的测试框架 H 在评估前被冻结。

演化（研究问题 2）。创建者从其自身冻结的研究问题 1 代码测试框架 H_0 开始。在开发过程中，它会收到来自固定的 100 个 SWE-Pro 反馈集任务和全部 89 个 Terminal-Bench 任务的结果。这 100 个 SWE-Pro 任务是创建阶段所用 731 个实例公开划分的子集，而第 4.3 节中的 630 个实例保留划分也来自同一划分。在所报告的协议中，控制器首先在两个基准上评估 H_0 。每个官方的 H_0 后候选版本随后被冻结，并作为一对提交：同一提交的一次完整的 100 个 SWE-Pro 任务评估和一次完整的 89 个 Terminal-Bench 任务评估。候选版本只有在两条评估都完成后才进入官方轨迹。同一提交的基础设施修复会被合并；探测、部分评估、已停止的运行和无效实例均被排除。

控制器提供十对 H_0 后完整评估的预算。在两次计费评估对之间，创建者

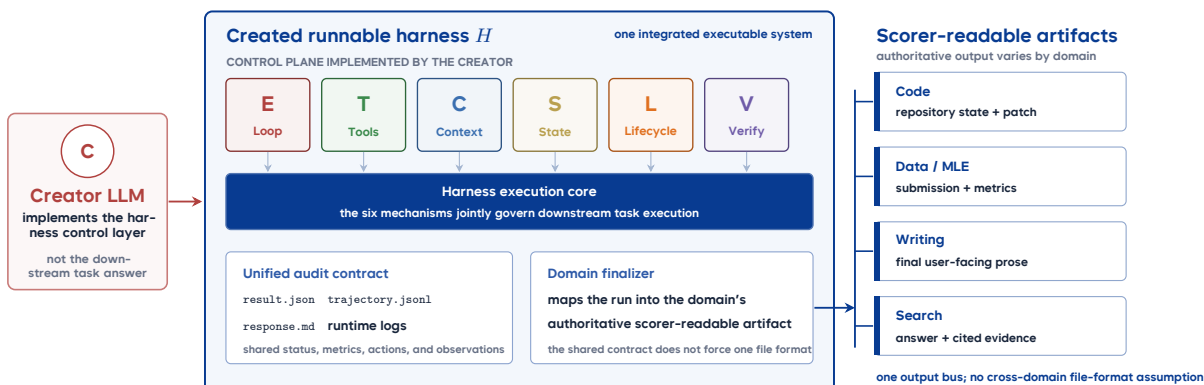


图 3 从控制层到可评分工件。创建者必须实现六个控制模块（E/T/C/S/L/V，颜色如图 1 所示）。完成的测试框架通过统一的、可审计的输出进行报告，并交付一个领域特定的、评分器可读的最终工件。

最多可以使用两次固定子集探测，每次探测覆盖两个基准中相同的前五个任务。探测结果仅用于诊断，永远不会成为官方评分。创建者通过声明一个具有完整官方评估对的非 H_0 提交来终止。

演化过程中展示的两个基准均为带有反馈的开发集，因此轨迹内得分衡量的是在线适应与版本选择。泛化能力在冻结后单独测量：每个官方版本还会在与反馈集不相交的 630 个 SWE-Pro 实例上进行评估，这些得分从不展示给创建者。在本文中，保留集指从创建者的开发循环中隐去的部分；第 4.3 节给出完整设置。

3.3 Domains and downstream benchmarks

Creation covers four domains and five downstream benchmarks (Table 2); Evolution currently focuses on code harnesses. Together, the suites contain 2,207 unique downstream instances. The Evolution feedback tasks come from the same benchmark suites and are therefore not counted again.

Domain	Benchmark	Tasks	Primary metric
Code	SWE-bench Pro [12], public split (abbreviated SWE-Pro below)	731	Task success
Code	Terminal-Bench 2.1 [45]	89	Task success
Data analysis	MLE-bench [7]	75	Medal score
Writing	EQ-Bench3 [37]	46	Rubric score
Research	BrowseComp [52]	1,266	Accuracy

表 2 创建阶段的下游评估覆盖范围。创建阶段使用 SWE-bench Pro 公开划分的全部 731 个实例。演化阶段从同一公开划分中抽取其 100 个任务的 SWE-Pro 反馈集和 630 个实例的保留划分，并使用全部 89 个 Terminal-Bench 任务作为反馈。

成熟的开源系统定义了各领域的能力边界。根据可用性，一个系统可以作为人工设计的参考，或提供开发环境。这些角色分别指定；纳入并不表示该系统同时承担两种角色。附录 A 列出了候选系统，而基准发布版本固定了它们的角色、版本和许可证。

3.4 评估协议

每个得分均由标准化运行时中的冻结测试框架产生。执行器大语言模型 L_E 和评估器 J 在每次比较中保持固定，因此得分变化反映的是测试框架的变化。开发与评估也相互分离：创建阶段不返回隐藏得分，演化阶段在开发期间仅暴露其指定的反馈集。

创建阶段。我们将创建的测试框架与公共种子以及（在可用时）成熟的人工设计测试框架进行比较。自评估设置 $L_E = L_C$ 并测量完整的创建者-测试框架系统。统一评估使用相同的固定 L_E 运行每个生成的测试框架，使各测试框架可直接比较。

演化阶段。演化候选在开发期间于指定的反馈基准上进行评估。只有完整的双基准对才进入官方轨迹，创建者选择一个最终的成对候选。所有轨迹结束后，每个官方版本还会在一个从未展示给创建者的不相交保留集上进行额外评估，因此对已观察反馈的适应与保留集泛化分别报告。附录 D 详细说明了评估设置和模型角色。

约束合规性。创建者可见的规范说明了提交的测试框架不得做什么：硬编码特定实例的解决方案、从任务标识符推导补丁、文件名允许列表或已知答案、查阅隐藏测试、隐藏答案、隐藏补丁、私有评分器内部信息或官方评估反馈，或用其自身的 LLM 访问路径替换所提供的与提供商无关的运行接口。两个特性

使这些约束可检查而非仅具建议性。首先，评分路径与测试框架隔离：测试框架自报的状态永远不会作为评分输入，SWE-Pro 的得分仅来自任务工作目录中留下的真实仓库差异，而 Terminal-Bench 的得分仅来自最终环境状态，因此任何测试框架都无法通过声称成功来获得分数。其次，每次运行都会保留其轨迹、结果和度量工件，并与冻结的测试框架源代码一起保存，这支持对所交付代码及该代码实际执行内容的事后审计。我们审计了本文报告的所有运行所交付的测试框架源代码和记录的执行工件，并将结果报告为无效结果：没有任何测试框架通过被禁止的途径获得分数，也没有任何运行因此被排除。

3.5 度量

对于每个冻结的测试框架，我们报告两个量：在基准原生度量下的下游任务性能，以及评估期间消耗的执行器模型令牌。执行成本以总令牌数和每任务平均令牌数报告；创建者用于构建或修改测试框架的令牌不计入。创建将每个测试框架与弱种子和可用的人工工程参考进行比较。演化报告其在相同执行器和评分器下相对于初始测试框架 H_0 的性能变化。

4 实验

我们实例化第 3 节定义的基准，并报告创建、演化以及跨模型行为的结果。行为分析比较资源使用、对反馈的响应以及跨执行器的迁移。

4.1 实验设置

我们评估六个创建者 LLM L_C ：Opus 4.8 [4]、GPT-5.5 [33]、Gemini 3.1 Pro [13]、DeepSeek V4 Pro [11]、Qwen 3.7 Max [1] 和 Seed 2.0 Pro [6]。模型通过其官方 API 或 OpenRouter [35] 运行。我们使用 Claude Code 2.1.177 作为开发环境 D ，但 GPT-5.5 使用 Codex 0.144.3。附录 B 给出了模型、解码和开发环境配置、下游执行资源和时间限制，以及人工参考来源。

我们遵循第 3.2 至 3.4 节中的开发与评估协议。针对研究问题一，我们为每个创建者与基准组合独立构建并评估三个测试框架，并报告平均前三名成绩。在自评估模式下，创建者同时担任执行者；在统一评估模式下，所有测试框架均使用双子座 3.1 专业版，从而隔离执行者兼容性的影响。人类参考结果采用已验证的公开系统结果，而非单一执行者下的配对对照；其来源列于附录 B.2。

4.2 测试框架创建

研究问题一探讨模型能否将可运行的弱种子转化为适用于任务族的有效测试框架。我们将生成的测试框架与得分为零的种子以及经过验证的人工工程系统进行比较，同时报告下游性能与执行令牌消耗。

总体发现。不同任务族之间的创建质量差异显著（表 3 至表 4 及图 4）。在自评估模式下，作品 4.8 取得最高总分（67.8），但仍低于人工工程参考值（86.2）。写作类测试框架接近参考水平，而搜索类差距最大，代码类同样落后。作品 4.8 与双子座 3.1 专业版在机器学习基准测试中领先，奖牌率分别为 32.9 和 32.4。更广泛地看，77.8% 的数据任务失败归因于测试框架缺陷，表明瓶颈不仅在于执行者能力。

性能波动与执行者依赖性。同一模型的独立创建结果仍可能存在显著差异。最典型的例子是一个作品代码测试框架，在自评估模式下表现良好，但在双子座环境下几乎崩溃，原因是其围绕原始执行者硬编码了 120 步的限制。数据类任务中过于严格的停止规则也会引发类似失败。因此，单个生成的测试框架不具备代表性，这促使我们报告平均前三名成绩。

更换执行者会显著改变排名。在双子座环境下，通义千问、种子与深度求索在若干数据与搜索设置中有所提升，而作品与 GPT-5.5 在使用自身执行者时往往表现更强。因此，自评估反映了测试框架设计、执行者能力以及二者之间的兼容性。成本同样不均衡：机器学习基准测试的令牌消耗相差约十九倍，但更高的成本并不必然带来更高的分数（图 9）。

Creator (Self-Eval)	SWE-Pro		Term.-2.1		MLE-bench		EQ-Bench3		BrowseComp		Avg.
	succ.	tok.	acc.	tok.	medal	tok.	score	tok.	acc.	tok.	score
<i>Seed harness H_{seed}</i>	0.0	—	0.0	—	0.0	—	0.0	—	0.0	—	0.0
Opus 4.8 High	69.3	759.5	64.8	46.2	32.9	101.7	84.6	3.1	52.4	593.9	67.8
GPT-5.5 High	32.8	325.6	52.1	12.2	19.1	29.3	83.0	4.9	52.6	313.5	55.1
Gemini 3.1 Pro High	43.6	783.6	68.8	177.1	32.4	131.3	74.8	5.2	35.2	267.4	55.6
DeepSeek V4 Pro High	28.9	739.7	35.6	35.2	19.6	208.4	75.4	4.9	40.9	1,449.9	45.2
Qwen 3.7 Max	33.5	421.9	41.3	19.3	3.1	42.9	68.7	4.3	32.3	130.9	44.0
Seed 2.0 Pro High	10.8	454.4	6.0	23.0	5.3	557.1	71.1	9.0	3.2	610.2	22.8
<i>Human harness + paired model</i>	80.0*	—	88.8*	—	24.0	66.5	83.7	9.2	92.2*	—	86.2

表 3 研究问题 1 / 自评估下的测试装置创建。除非另有说明，每位创建者独立构建并评估三个测试装置，分数为平均@3。每个基准使用其原生度量，tok. 是每个测试装置的平均执行令牌数（以百万计），Avg. 是 SWE-Pro、Terminal-Bench、EQ-Bench3 和 BrowseComp 的未加权平均值。MLE-bench 涵盖 33 个物理单元和 2,475 个结果。人类行是系统级参考；* 标记我们在此实验中未重新运行的外部结果，— 标记不可用条目。

Creator (Unified-Eval)	SWE-Pro		Term.-2.1		MLE-bench		EQ-Bench3		BrowseComp		Avg.
	succ.	tok.	acc.	tok.	medal	tok.	score	tok.	acc.	tok.	score
<i>Seed harness H_{seed}</i>	0.0	—	0.0	—	0.0	—	0.0	—	0.0	—	0.0
Opus 4.8 High	33.0 [‡]	782.9	52.4	94.3	16.9	46.3	74.2	2.1	53.6	505.8	53.3 [‡]
GPT-5.5 High	27.8	383.9	49.4	24.3	16.0	80.5	46.5	4.9	55.4	177.8	44.8
Gemini 3.1 Pro High	43.6	783.6	68.8	177.1	32.4	131.3	74.8	5.2	35.2	267.4	55.6
DeepSeek V4 Pro High	29.2 [‡]	492.4	38.2 [‡]	26.8	9.8	125.5	72.9	5.0	54.8	362.8	48.8 [‡]
Qwen 3.7 Max	41.3	1,253.3	48.6	81.9	16.0	44.0	71.5	2.5	49.9	133.4	52.8
Seed 2.0 Pro High	15.6	786.8	13.1	79.0	13.3	1,717.5	73.1	44.9	17.3	398.4	29.8

表 4 研究问题 1 / 固定 Gemini 执行器下的测试装置创建。每个创建者的测试装置均由 Gemini 3.1 Pro 执行。Gemini 行复用了自评估控制组，不计为新的物理单元。每个分数条目均为平均@3。标记为 [‡] 的代码单元包含一个折叠的 R3 副本；移除它后，Opus 在 SWE-Pro 上的事后干净敏感性均值为 49.1，DeepSeek 在 SWE-Pro/Terminal-Bench 上为 43.8/57.3。GPT-5.5 的 EQ-Bench3 单元为平均@3（46.5）；排除其值为零的第一个测试装置后，均值为 69.7。所有其他定义遵循表 3。

实现行为。六位创建者遵循不同的实现策略。Opus 经常重写执行栈；GPT-5.5 添加一个大型单体代理；DeepSeek、Qwen 和 Seed 通过代理、工具、上下文和状态模块扩展种子；Gemini 主要就地编辑运行器。18 个代码工件总共添加了 17,111 行净代码（表 5），但编辑规模并不能预测性能。Gemini 添加的行数最少（1,006），却获得了最佳的 Terminal-Bench 分数（68.8），这表明专注的更改和频繁的验证比代码量更重要。

所有 18 个代码测试装置都实现了显式执行循环；工具、生命周期控制和验证分别在 13/18、13/18 和 15/18 个工件中完整实现（图 5）。状态和内存是最明显的缺口：11/18 个工件定义了状态类，但只有一个暴露了状态保存接口，且只有一个实现了周期性检查点。在 26,679 条记录的任务轨迹中未出现任何检查点事件。另一个反复出现的弱点是执行器特定配置：硬编码的步骤或输出限制可能使功能正常的测试装置在执行器更改时失效。验证也大多是语法层面的；2,325 个已执行的数据任务中有 441 个产生了退化提交，而没有任何测试装置检测到。

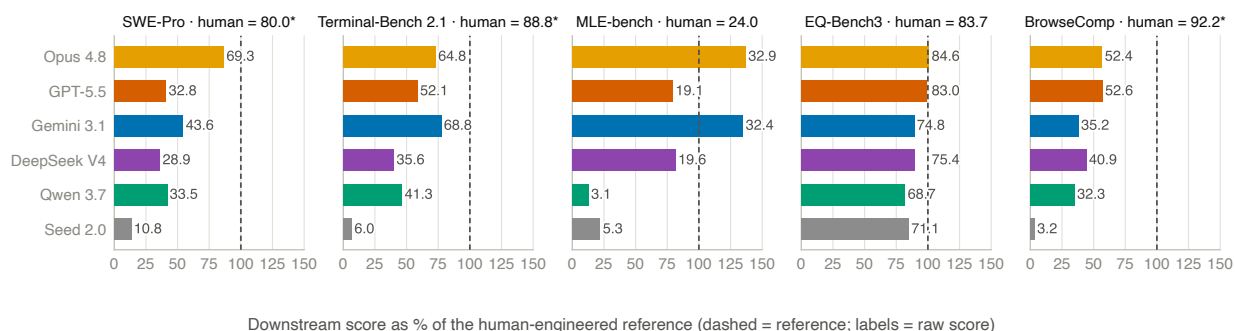


图 4 研究问题一：与人工设计参考系统的距离。每位创建者的自评下游得分按该基准的人工设计参考进行归一化；虚线为参考线，柱旁标签为原始得分。人工参考来自不同的测试框架-模型组合，并非同一执行器下的配对对照，因此本图仅展示与所选成熟系统的距离。超过 100% 意味着超过该外部参考，而非超过人类能力。

创建者文件 新增/修改/删除	总净代码行数 中位数/副本	范围	SWE-Pro	Term.-2.1		
Opus 4.8	19/7/16	2,470	698	656–1,116	69.3	64.8
GPT-5.5	3/10/0	3,537	1,231	1,059–1,247	32.8	52.1
Gemini 3.1 Pro	4/4/0	1,006	324	270–412	43.6	68.8
DeepSeek V4 Pro	14/4/1	3,242	988	931–1,323	28.9	35.6
Qwen 3.7 Max	12/6/0	3,562	1,339	551–1,672	33.5	41.3
Seed 2.0 Pro	14/6/0	3,294	1,200	868–1,226	10.8	6.0

表 5 冻结的研究问题一代码工件的编辑规模。文件计数和总净代码行数按每位创建者的三个独立创建工作件求和；中位数和范围按单个工件计算。统计仅涵盖创建者负责的 harness/目录，不包括运行器注入的运行时基板。性能为自评平均@3。

部分生成的机制从未影响执行。在代码的 108 个组件实例中，72 个在实际运行中触发，18 个仅有部分证据，18 个从未被观察到；所有未观察到的实例均涉及状态和内存。同样的模式也出现在代码之外：写作的 587 个特征中有 124 个被确认为死代码，数据的 36 个机制位于死路径上。微小的实现错误也可能使整个工具链失效。仅靠自测数量是一个弱信号：其与下游得分的斯皮尔曼相关性仅为 0.13–0.26 且不显著，而修订调用达到 0.57 ($p \leq .0005$)。当创建者阅读失败信息、做出针对性修改并重新验证时，测试才有效。

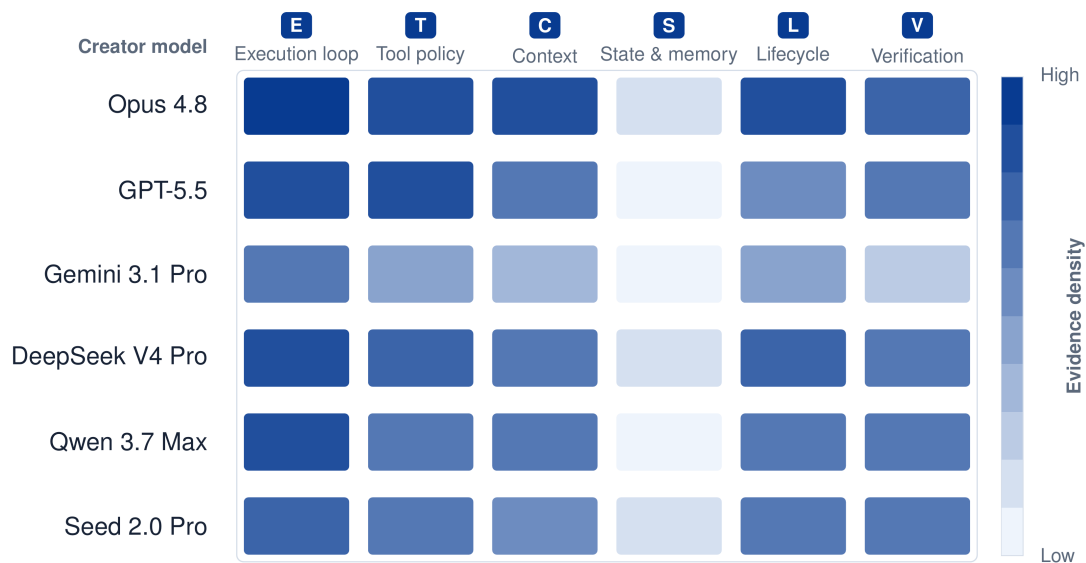


图 5 跨领域和阶段的测试框架架构证据。六列分别为执行循环、工具策略、上下文管理、状态与内存、生命周期与恢复以及结果验证。每个领域首先按实际可用的独立测试框架数量进行归一化，然后在研究问题一的代码/数据/写作/搜索和研究问题二的主干上聚合；种子没有研究问题二的轨迹，因此仅由四个研究问题一领域构建。完整权重要求机制进入主路径或在正式运行中触发，而声明的代码、配置或瞬态状态仅获得部分权重。颜色表示机制证据密度，而非得分、显著性水平或因果效应。

执行器迁移。可移植性取决于各个测试框架（图 6）。多个千问（Qwen）和深度求索（DeepSeek）测试框架在双子座（Gemini）执行器下有所改进，表明其原始执行器是瓶颈；千问在浏览综合基准（BrowseComp）上提升 17.6 分，在机器学习基准（MLE-bench）上提升 12.9 分。作品（Opus）呈现相反模式：其自评软件工程专业基准（Self-Eval SWE-Pro）得分在双子座执行器下从 69.3 降至 33.0，写作得分从 84.6 降至 74.2。在作品搜索测试框架中，执行器更换后重复查询率从 10.1% 升至 88.2%，表明其去重、审查和终止规则是针对原始模型调整的。因此，可运行的测试框架可被其他模型使用，但能力迁移仅在其提示、工具协议、预算和停止规则保持兼容时发生。

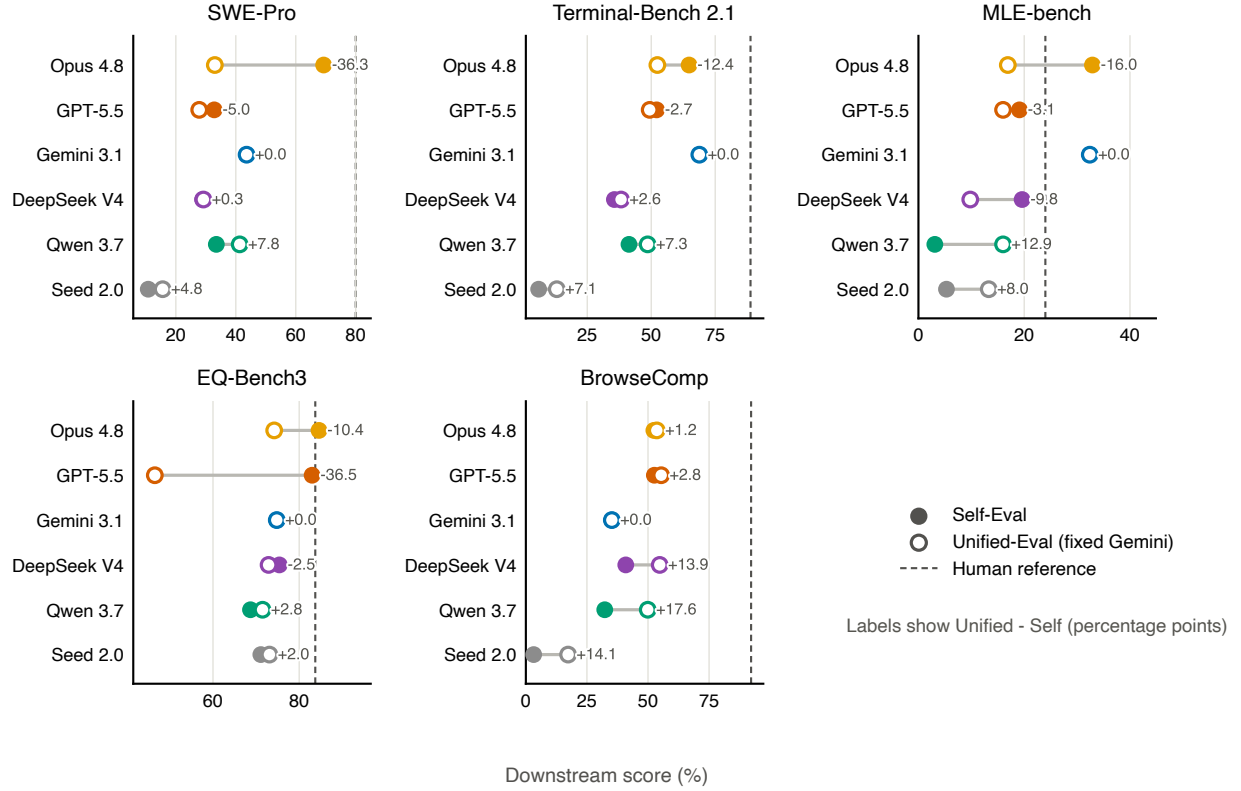


图 6 固定双子座执行器下的测试框架可移植性。实心标记为自评结果，空心标记为同一创建者测试框架在固定双子座执行器下的结果，标签为统一评估减去自评的差值，虚线为外部人工工程系统参考。所有面板均使用表 3 和表 4 的 avg@3 值；两个折叠代码单元的干净敏感性均值见表 4 图注。

4.3 测试框架演化

总体发现。所有五个自运行时创建者在可见反馈对上均有改进，但在留出任务上增益缩小。作品 4.8 在留出任务上改进最大，为 +4.44 分。在固定双子座执行器下迁移较弱：仅作品在留出任务上有所改进，其他三个谱系均出现退化。因此，演化可以产生有用的局部变化，但增益仍然较小，且通常针对当前执行器或反馈集特化。

评估协议。研究问题 2 (RQ2) 探究创建者能否利用下游执行反馈改进其研究问题 1 (RQ1) 代码测试框架。演化期间反复评估的任务构成反馈集；仅在演化后评估且结果从不返回给创建者的任务构成留出集。我们报告单个基准得分和等权配对得分：

$$\bar{P}_t = \frac{1}{2} (P_t^{\text{SWE100}} + P_t^{\text{Term89}}), \quad (2)$$

其中两项为百分比得分， t 索引完成正式评估的冻结版本。

每个谱系从其 RQ1 测试框架 H_0 开始。我们运行五条自运行时轨迹和四条固定 Gemini 消融轨迹，使用相同的创建者、开发环境和起始测试框架。如第 3.2 节所定义，官方版本必须完成 100 项任务的 SWE-Pro 和 89 项任务的 Terminal-Bench 评估；探测仅用于诊断。九个谱系产生 73 个官方版本和 64 次相邻版本切换；图 7 展示了每条反馈集轨迹。

所有轨迹结束后，我们在 630 个实例的 SWE-Pro 留出集上评估每个官方版本，该留出集来自与 100 项任务反馈集不相交的公开分割实例（第 3.2 节）。这些分数从未展示给创建者，也不会影响编辑、停止或最终版本选择。因此，表 6 将可见的反馈增益与留出泛化分开，图 8 为每个谱系叠加了这两条轨迹。

Setting	Creator	Feedback pair	Held-out-630	Held-out final gap
		$H_0 \rightarrow H_{\text{dec}}$	$H_0 \rightarrow H_{\text{dec}}$	
Self	Gemini 3.1 Pro	59.9 $\rightarrow$ 68.7 (+8.8)	48.89 $\rightarrow$ 51.59 (+2.70)	0.00
Self	Opus 4.8	71.1 $\rightarrow$ 74.1 (+3.0)	63.02 $\rightarrow$ 67.46 (+4.44)	1.59
Self	Qwen 3.7 Max	41.8 $\rightarrow$ 55.7 (+13.9)	42.22 $\rightarrow$ 43.65 (+1.43)	3.17
Self	DeepSeek V4 Pro	47.2 $\rightarrow$ 60.6 (+13.4)	47.30 $\rightarrow$ 50.48 (+3.17)	1.75
Self	GPT-5.5	59.2 $\rightarrow$ 65.1 (+5.9)	48.25 $\rightarrow$ 52.06 (+3.81)	0.00
Fixed Gemini	Opus 4.8	58.8 $\rightarrow$ 68.6 (+9.7)	48.10 $\rightarrow$ 50.79 (+2.70)	2.54
Fixed Gemini	Qwen 3.7 Max	62.1 $\rightarrow$ 63.2 (+1.1)	49.52 $\rightarrow$ 48.41 (-1.11)	1.11
Fixed Gemini	DeepSeek V4 Pro	47.3 $\rightarrow$ 53.8 (+6.5)	43.02 $\rightarrow$ 40.63 (-2.38)	3.02
Fixed Gemini	GPT-5.5	56.6 $\rightarrow$ 59.1 (+2.4)	42.22 $\rightarrow$ 31.90 (-10.32)	16.51

表 6 RQ2 反馈集增益与冻结后留出 630 泛化。反馈对是 SWE-Pro-100 和 Terminal-Bench-89 百分比分数的未加权平均值；留出列仅使用之后运行的 SWE-Pro-630 任务。最终差距是谱系的最佳留出分数减去其声明版本的分数。Gemini 对照组仅列出一行。

编辑与反馈使用。声明版本的中位数更改八个文件，添加 476 行并删除 38 行（表 7）。在 64 次官方切换中，58 次更改执行或控制流，37 次更改工具，17 次更改生命周期恢复，16 次更改上下文，仅四次更改状态；没有切换修改独立验证器。编辑大小不能可靠预测改进，同一创建者在不同执行器下可能采用不同策略。例如，DeepSeek 扩展了其自运行时测试框架，但在上下文压缩破坏工具消息配对后，又回滚了大部分固定 Gemini 重写。因此，演化类似于围绕运行时反馈的局部程序搜索，其中删除可能与添加同样有用。

Setting	Creator	Official switches	$H_0 \rightarrow H_{\text{dec}}$	cumulative diff	Main edit focus
Self	Gemini 3.1 Pro	8	11 files, +692/ - 38		Tool output, editing, timeouts, patch cleanup
Self	Opus 4.8	3	8 files, +450/ - 18		Completion gate, self-review, process recovery
Self	Qwen 3.7 Max	8	9 files, +1101/ - 106		Empty-response/parse recovery, auto-submit checkpoints
Self	DeepSeek V4 Pro	10	9 files, +1730/ - 58		Completion logic, tool extensions, workspace pre-analysis
Self	GPT-5.5	7	4 files, +496/ - 49		Final-state review gate and artifact tracking
Fixed Gemini	Opus 4.8	4	3 files, +227/ - 21		Pre-completion verification, scratch-file cleanup
Fixed Gemini	Qwen 3.7 Max	5	4 files, +219/ - 96		Context and message-sanitizer rewrite
Fixed Gemini	DeepSeek V4 Pro	9	8 files, +476/ - 12		Message-protocol recovery, workspace discovery
Fixed Gemini	GPT-5.5	10	6 files, +297/ - 34		Probes, branch rollback, final-state review

表 7 九个 RQ2 谱系的代码编辑大小。“官方切换”仅计算相邻官方版本；累积差异比较 H_0 与创建者为每个谱系声明的版本。编辑焦点描述的是该单条轨迹中观察到的行为，并非相应创建者模型的普遍属性。

九个谱系中有八个至少完成了一个从读取结果到编辑、再评估和版本选择的完整循环。故障诊断仍然是最薄弱的环节：专用轨迹接口仅被调用两次，明确检查的案例仅覆盖 189 个反馈任务中的 0.5% - 40.2% 个，具体取决于谱系。创建者转而依赖自定义脚本和小型探针，这两者都可能与完整评估不一致；一个 GPT-5.5 候选通过了所有五个终端探针，但在完整集合上仅得 0.584 分。

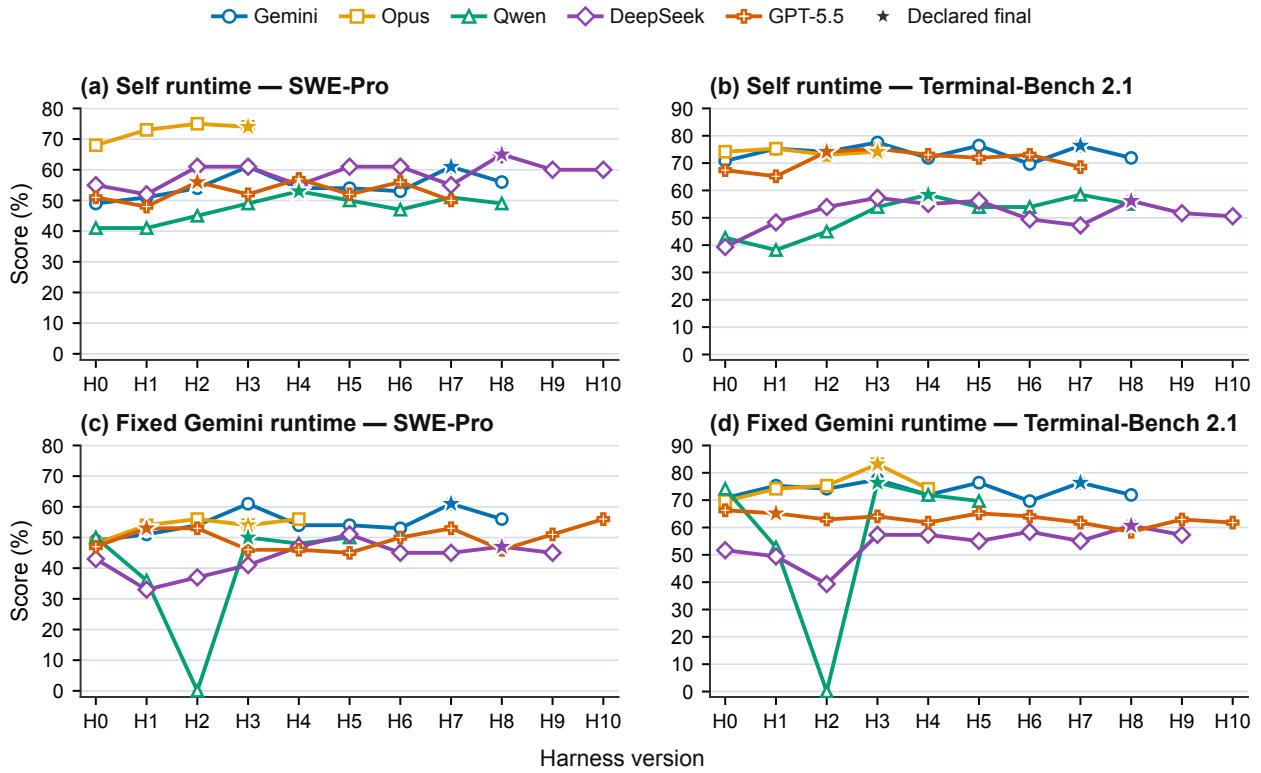


图 7 SWE-Pro 和 Terminal-Bench 上的反馈集演化轨迹。顶行为自身运行时，底行为固定的 Gemini 运行时；左列为 SWE-Pro-100，右列为 Terminal-Bench-89。H₀ 是冻结的 RQ1 测试框架，H_i 是在两个基准上完成正式评估的后续冻结提交，星号标记创建者声明的版本。Gemini 对照在两个设置中重复使用，每个创建者-运行时单元有一条轨迹。

Opus 给出了最清晰的正面例子：它发现 100 次运行中有 99 次报告成功但只有 48 次通过，将差距追溯到过早完成，并添加了完成检查。当反馈暴露具体失败模式且由此产生的更改得到端到端验证时，反馈最为有用。

稳定性与最终版本选择。演化并非单调。在 64 次官方切换中，八次在两个基准上均出现退化，16 次出现单基准退化，三次出现跨基准权衡，七次未产生可测量的变化，27 次报告的增益仍处于重复运行噪声带内，两次有超出噪声带的明确正面证据，一次不包含可执行代码更改。同一提交可能相差约 ± 4.75 个配对分数点，因此仅凭分数无法将小幅增益归因于代码更改。新增代码也未必活跃：在 169 个新函数或类中，113 个可从入口点到达，31 个仅通过死代码可达，25 个没有调用者。Opus 的完成门控是一个具有路径和案例级证据的正面例子；Qwen 的消息清理器则相反，破坏了有效的 Gemini 工具结果序列。

创建者通常选择接近最佳可见反馈分数的版本，但该选择很少与最佳留出版本一致。所有五种自运行时声明在留出任务上均优于 H₀，增益为 +1.43 至 +4.44 分，平均增益为 +3.11 分。然而，在固定双子座（Gemini）条件下，仅作品（Opus）有所改进，其余三者均出现退化。在 64 次可比切换中，反馈分数与留出分数同向变动仅 34 次（53.1%），且仅 2/9 的声明版本为留出最优。因此，可见反馈对局部搜索有用，但对最终选择不可靠：反复优化噪声分数可能偏向幸运运行并放大过拟合。

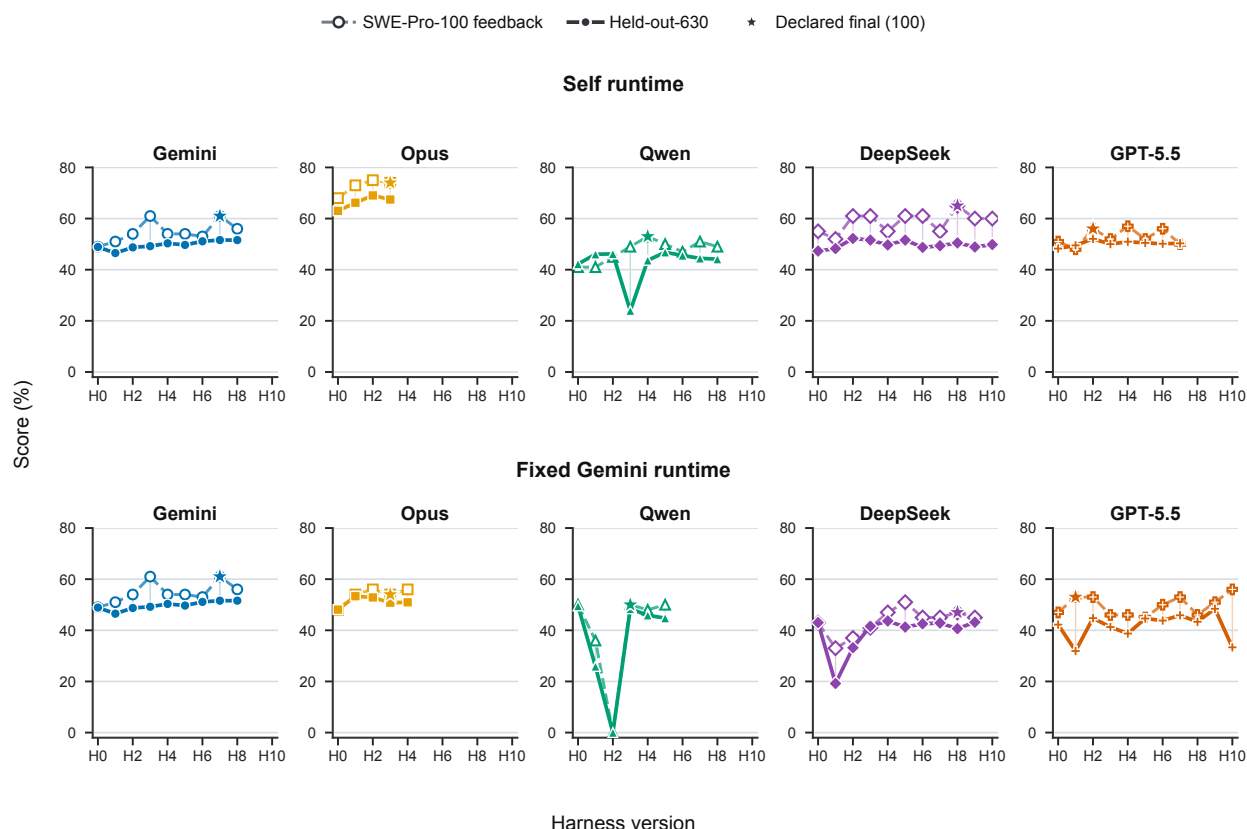


图 8 SWE-Pro-100 反馈轨迹与冻结后留出-630 性能。每列隔离一个创建者模型，并将其可见的 100 任务反馈轨迹与冻结后的 630 任务留出轨迹叠加。顶行为自运行时，底行为固定双子座（Gemini）。星号标记从 100 任务反馈中声明的最终版本。淡色垂直直线段连接匹配的测试装置版本。双子座（Gemini）对照仅为视觉比较而重复。

5 相关工作

测试装置开发（HarnessDev）位于智能体基准、自主智能体构建与测试装置演化的交汇处。

智能体基准与测试装置开发。诸如 SWE-bench[18]、GAIA[29]、WebArena[61]、 τ -bench[54] 和 AgentBench[26] 等基准标准化了任务、环境和评分，但通常在选定测试装置下评估任务执行。相反，测试装置基准（Harness-Bench）[55] 衡量测试装置选择如何改变模型性能。元智能体挑战（Meta-Agent Challenge）[27] 直接评估开发过程：元智能体在沙箱中迭代编程智能体工件，并在五个领域的受保护留出测试上评分。它与创建（Creation）密切相关，而测试装置开发（HarnessDev）还研究持续开发、分离创建者与执行者模型，并衡量执行成本。

HarnessOpt-Bench [47] 是与 Evolution 最接近的同期基准。一个 LLM 优化器接收种子 harness、分级反馈和固定的评估预算；随后，一个可信环境通过在不可访问的测试分区上的归一化增益来对其提名的候选方案进行评分。其重点是优化所提供的 harness，尽管其近乎空的 GAIA 种子也需要构建。相比之下，HarnessDev 将从头开始的 Creation 与 Evolution 连接起来，并在自运行时和固定运行时模型下评估相同的冻结工件。它使用轨迹内的分数作为反馈，然后在不相交的 630 任务 SWE-Pro 集上测试每个冻结版本，从而在整个轨迹中将适应与留出泛化分离开来。

Evo-Bench [16] 也通过要求进化模型改进共享的 CodeAct 种子来评估 Evolution，同时保持运行时模型固定。它在一个不相交的、经过敏感性校准的多领域套件上对每个谱系的最终修订版进行评分。相比之下，HarnessDev 连接了 Creation 和 Evolution，包括自运行时和固定运行时视图，测量执行令牌成本，评估向另一个执行器的迁移，并在其留出分割上对每个冻结版本进行评分。因此，Evo-Bench 强调跨领域的最终修订质量，而 HarnessDev 还研究留出性能如何沿着反馈驱动的轨迹变化。

自动化智能体设计与进化。人类设计的系统，如 Claude Code [2]、Codex [32]、OpenHands [49] 和 SWE-agent [53]，结合了执行循环、工具、上下文管理和恢复机制 [40]。ADAS [15]、AFlow [57]、MASS [60] 和 EvoAgentX [51] 在提示、工作流、算子或智能体拓扑上进行搜索；其他系统则让模型用自然语言或代码构建更多的智能体 [22, 31, 39]。

近期系统直接优化可执行测试框架代码。VeRO [46] 提供版本化快照、预算受控评估和结构化轨迹。元测试框架（Meta-Harness）[21] 采用编码智能体提议器，在提出下一个测试框架之前可检查先前候选、评分和轨迹。其他方法在回合之间改进提示、工作流、记忆、技能、工具或代码 [10, 14, 19, 20, 23, 25, 28, 42, 43, 48]。自测试框架（Self-Harness）[56] 从失败中推导模型特定编辑并应用回归测试；测试框架修复（HarnessFix）[9] 使用来源和控制流感知表示进行局部修复；演示进化（DemoEvolve）[8] 在奖励反馈不可靠时使用演示；测试框架指南针（HarnessCompass）[58] 解决过拟合和组件干扰问题。测试框架 R1（Harness-R1）[41] 则训练一个测试框架工程师将失败批次转换为经过验证的补丁。这些工作提出了测试框架改进的方法或基础设施；测试框架开发（HarnessDev）评估通用前沿模型在单一创建与进化协议下执行更广泛开发者角色的能力。

评估测试框架进化。最终任务增益可能混淆知情诊断与盲目搜索。优先级排序评估 [17] 测试优化器是否识别最值得更改的组件。测试框架更新并非测试框架收益 [24] 将产生有用更新与执行器利用该更新的能力分开，这启发了我们的创建者-执行者分离以及自评估 / 统一评估视角。SEAGym [59] 记录中间快照、成本以及分布内和分布外结果，表明后续更新不必保留留出增益。匹配预算研究同样发现测试框架进化可能过拟合其搜索基准，并且可能无法超越更简单的搜索基线 [50]。因此，测试框架开发（HarnessDev）冻结可运行工件，记录开发轨迹和执行成本，并评估跨运行时模型的迁移。我们将其进化轨迹视为对带有反馈集合的适应，并随后在不相交的 SWE-Pro 任务上评估留出泛化能力；匹配搜索评估仍属未来工作。

6 讨论、局限性与结论

结果表明为何应直接评估执行框架的开发。创建性能在不同领域差异显著：在自我评估下，当前模型在写作方面与人类参考水平相当，在机器学习实验方面超越人类参考，但在搜索和研究方面仍远远落后，在代码方面也仍落后于人类参考。跨执行器比较显示，某些执行框架在更强的执行器下有所改进，而另一些则表现出创建者协同适应。演化则更为困难：有用的中间更新往往被后续更改抹去，且更多更新并不能保证最终获得正向收益。综合来看，这些发现将持久执行系统的质量与运行于其中的模型能力区分开来。固定双子座演化消融实验进一步印证了这一点：仅改变运行时绑定就能显著移动 H_0 ，并改变哪些执行框架更改是有用的。

6.1 局限性

这四类覆盖了许多但并非所有实际部署。人类基线并不均衡，且不能保证最优。统一评估减少但无法完全消除执行器与模型之间的差异，因为执行框架与模型的交互十分复杂。行为比较是描述性的，并受限于基准覆盖不完整。演化目前每个创建者-运行时单元仅有一条轨迹，且有一个未完成的主运行时单元，其冻结后的保留评估仅覆盖软件工程专业版，因此这些轨迹无法支持不确定性

估计或群体层面的比较。开发环境 D 在两个阶段中均保持不变；演化后的执行框架本身能否作为进一步演化的开发环境，留待未来工作。最后，执行框架开发衡量的是模型外部学习，并不声称启发式学习可以取代参数训练。

6.2 结论

执行框架开发将智能体评估从模型能否在固定系统内解决任务，转向其能否创建并维护解决未来任务的系统。通过四类人类基线语料库、从零开始的创建任务、反馈驱动的演化、自我评估与统一评估，以及与人类工程参考的比较，它使智能体构建的执行框架成为可测量的对象。如果说模型权重是智能积累的一个场所，那么执行框架则是另一个：显式、可检查、可测试、可复用，并能在失败、反馈和真实工程压力下持续改进。

伦理声明。HarnessDev 基于公开可用的基准套件和开源测试框架构建；未收集任何人类生成的内容。自动化测试框架构建存在放大不安全工具使用的风险，因此该基准对提交的测试框架可执行的操作设定了明确约束，并在每次运行后审计合规性（第 3.4 节）；本研究中未观察到违规行为，我们发布了审计工件，以便重复检查。创建运行和下游基准任务在容器中执行，但该边界是为可复现性而设置，而非用于隔离；任何重用所生成测试框架的人员都应将其视为不受信任的代码，并比我们更严格地加以隔离。

7 贡献

核心贡献者 吴宇浩、张景源、石佳俊

贡献者 雷新平、顾清水、张宇轩、王泽轩、何晨、黄晨、宋茂嘉、曾志远、王少文、刘金凯、石云峰、刘佳恒

通讯作者 吴宇浩 (
wuyuhao.621@bytedance.com) 严深
(sheny@bytedance.com) 黄文浩
(huang.wenhao@bytedance.com) 张戈
(gezhang@umich.edu) 张文轩
(wxzhang@sutd.edu.sg)

参考文献

- [1] Alibaba Group. Qwen3.7-max. <https://www1.hkexnews.hk/listedco/listconews/sehk/2026/0520/2026052001340.pdf>, 2026. Official announcement.
- [2] Anthropic. Claude Code: an agentic coding assistant. <https://docs.anthropic.com/claude-code>, 2024.
- [3] Anthropic. Effective context engineering for AI agents. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>, 2025.
- [4] Anthropic. Claude opus 4.8. <https://www.anthropic.com/claude/opus>, 2026. Official model page covering Claude Opus 4.7 and 4.8.
- [5] browser-use contributors. browser-use. <https://github.com/browser-use/browser-use>, 2026.
- [6] ByteDance Seed. Seed2.0. <https://seed.bytedance.com/en/seed2>, 2026.
- [7] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. MLE-bench: Evaluating machine learning agents on machine learning engineering. *arXiv:2410.07095*, 2024.
- [8] Lirong Che, Yuzhe Yang, Peiwen Lin, Chuang Wang, Xueqian Wang, and Jian Su. DemoEvolve: Overcoming sparse feedback in agentic harness evolution with demonstrations, 2026. URL <https://arxiv.org/abs/2605.24539>.
- [9] Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws, 2026. URL <https://arxiv.org/abs/2606.06324>.
- [10] Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, et al. Harnessx: A composable, adaptive, and evolvable agent harness foundry. *arXiv preprint arXiv:2606.14249*, 2026.
- [11] DeepSeek-AI. DeepSeek-V4 preview release. <https://api-docs.deepseek.com/news/news260424/>, 2026.
- [12] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. SWE-bench Pro: Can AI agents solve long-horizon software engineering tasks? *arXiv:2509.16941*, 2025.
- [13] Google. Gemini 3.1 pro: A smarter model for your most complex tasks. <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/>, 2026.
- [14] Yufei He, Juncheng Liu, Yue Liu, Yibo Li, Tri Cao, Zhiyuan Hu, Xinxing Xu, and Bryan Hooi. Evotest: Evolutionary test-time learning for self-improving agentic systems. *arXiv preprint arXiv:2510.13220*, 2025.
- [15] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems, 2024. URL <https://arxiv.org/abs/2408.08435>.
- [16] Lisheng Huang, Chen Yang, Hao Zhou, Huatong Song, Zongchao Chen, Ran Le, Yang Song, Wayne Xin Zhao, and Tao Zhang. Evo-Bench: Can language models improve agent harness?, 2026. URL <https://arxiv.org/abs/2608.09096>.
- [17] Kai Tzu iunn Ong, Minseok Kang, Dongwook Choi, Junhee Cho, Seungju Kim, Seungwon Lim, Geunha Jang, Minwoo Oh, Bogyoung Jeong, Sunghwan Kim, Taeyoon Kwon, and Jinyoung Yeo. Towards direct evaluation of harness optimizers via priority ranking, 2026. URL <https://arxiv.org/abs/2605.22505>.
- [18] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues?, 2023. URL <https://arxiv.org/abs/2310.06770>.
- [19] Seth Karten, Joel Zhang, Tersoo Upaa Jr, Ruihong Feng, Wenzhe Li, Chengshuai Shi, Chi Jin, and Kiran Vodrahalli. Continual harness: Online adaptation for self-improving foundation agents. *arXiv preprint arXiv:2605.09998*, 2026.
- [20] Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, and Yujin Tang. Recursive harness self-improvement, 2026. URL <https://arxiv.org/abs/2607.15524>.

- [21] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-Harness: End-to-end optimization of model harnesses, 2026. URL <https://arxiv.org/abs/2603.28052>.
- [22] Hongwei Li, Zhun Wang, Qinrun Dai, Yuzhou Nie, Jinjun Peng, Ruitong Liu, Jingyang Zhang, Kaijie Zhu, Jingxuan He, Lun Wang, Yangruibo Ding, Yueqi Chen, Wenbo Guo, and Dawn Song. Opensage: Self-programming agent generation engine, 2026. URL <https://arxiv.org/abs/2602.16891>.
- [23] Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, et al. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026.
- [24] Minhua Lin, Juncheng Wu, Zijun Wang, Zhan Shi, Yisi Sang, Bing He, Zewen Liu, Tianxin Wei, Zongyu Wu, Zhiwei Zhang, Dakuo Wang, Xiang Zhang, Benoît Dumoulin, Cihang Xie, Yuyin Zhou, Suhang Wang, and Hanqing Lu. Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving LLM agents, 2026. URL <https://arxiv.org/abs/2605.30621>.
- [25] Siwei Liu, Jinyuan Fang, Han Zhou, Yingxu Wang, and Zaiqiao Meng. Sew: Self-evolving agentic workflows for automated code generation. *arXiv preprint arXiv:2505.18646*, 2025.
- [26] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents, 2023. URL <https://arxiv.org/abs/2308.03688>.
- [27] Xinyu Lu, Tianshu Wang, Pengbo Wang, Zujie Wen, Zhiqiang Zhang, Jun Zhou, Boxi Cao, Yaojie Lu, Hongyu Lin, Xianpei Han, and Le Sun. The meta-agent challenge: Are current agents capable of autonomous agent development?, 2026. URL <https://arxiv.org/abs/2606.04455>.
- [28] Xiaotian Luo, Dizhan Xue, Fengxingyu Wang, Chuanrui Hu, and Yafeng Deng. Harnessbank: Semantic gene-bank search with gated verification for agent-harness self-evolution, 2026. URL <https://arxiv.org/abs/2607.13683>.
- [29] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for general ai assistants, 2023. URL <https://arxiv.org/abs/2311.12983>.
- [30] MIT Project NANDA. The GenAI divide: State of AI in business 2025. https://www.artificialintelligence-news.com/wp-content/uploads/2025/08/ai_report_2025.pdf, 2025. Industry report.
- [31] Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiayu Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, et al. Code as agent harness, 2026. URL <https://arxiv.org/abs/2605.18747>.
- [32] OpenAI. Codex CLI. <https://github.com/openai/codex>, 2025.
- [33] OpenAI. Introducing GPT-5.5. <https://openai.com/index/introducing-gpt-5-5/>, 2026.
- [34] OpenAI. GPT-5.6: Frontier intelligence that scales with your ambition. <https://openai.com/index/gpt-5-6/>, July 2026. Official release report and benchmark result tables.
- [35] OpenRouter. Openrouter quickstart guide. <https://openrouter.ai/docs/quickstart>, 2026.
- [36] Gergely Orosz. What are forward deployed engineers, and why are they so in demand? <https://newsletter.pragmaticengineer.com/p/forward-deployed-engineers>, 2025. The Pragmatic Engineer.
- [37] Samuel J. Paech. EQ-Bench 3: Emotional intelligence benchmark. <https://github.com/EQ-bench/eqbench3>, 2025. 46 scenarios; LLM-judged rubric and pairwise Elo evaluation.
- [38] Palantir Technologies. A day in the life of a Palantir forward deployed software engineer. <https://blog.palantir.com/a-day-in-the-life-of-a-palantir-forward-deployed-software-engineer-45ef2de257b1>, 2020.
- [39] Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-language agent harnesses, 2026. URL <https://arxiv.org/abs/2603.25723>.
- [40] Benjamin Rombaut. Inside the scaffold: A source-code taxonomy of coding agent architectures, 2026. URL <https://arxiv.org/abs/2604.03515>.

- [41] Shuai Shao, Kangning Zhang, Qingyao Li, Shijian Wang, Hao Wang, Wenxiang Jiao, Yuan Lu, Yi Guo, Weiwen Liu, and Weinan Zhang. Harness-R1: Learning to edit executable runtime harnesses from agent failure trajectories, 2026. URL <https://arxiv.org/abs/2608.02276>.
- [42] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>.
- [43] Wangcheng Tao, Han Wu, and Weng-Fai Wong. Sepo: Self-evolving prompt agent for system prompt optimization. *arXiv preprint arXiv:2606.04465*, 2026.
- [44] The New Stack. Why OpenAI and Anthropic are hiring forward deployed engineer teams. <https://thenewstack.io/forward-deployed-engineers-ai/>, 2026.
- [45] The Terminal-Bench Team. Terminal-bench: A benchmark for AI agents in terminal environments. <https://www.tbench.ai/>, 2026. Terminal-Bench 2.1 leaderboard, Laude Institute.
- [46] Varun Ursekar, Apaar Shanker, Veronica Chatrath, Yuan Xue, and Samuel Marc Denton. VeRO: A harness for agents to optimize agents, 2026. URL <https://arxiv.org/abs/2602.22480>.
- [47] Varun Ursekar, Apaar Shanker, Yash Maurya, Shehab Yasser, Vijay S. Kalmath, Veronica Chatrath, and Yuan Xue. HarnessOpt-Bench: Evaluating LLMs at harness optimization, 2026. URL <https://arxiv.org/abs/2608.06301>.
- [48] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. URL <https://arxiv.org/abs/2305.16291>.
- [49] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. Openhands: An open platform for ai software developers as generalist agents, 2024. URL <https://arxiv.org/abs/2407.16741>.
- [50] Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents, 2026. URL <https://arxiv.org/abs/2607.12227>.
- [51] Yingxu Wang, Siwei Liu, Jinyuan Fang, and Zaiqiao Meng. EvoAgentX: An automated framework for evolving agentic workflows, 2025. URL <https://arxiv.org/abs/2507.03616>.
- [52] Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. BrowseComp: A simple yet challenging benchmark for browsing agents. *arXiv:2504.12516*, 2025.
- [53] John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- [54] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL <https://arxiv.org/abs/2406.12045>.
- [55] Yilun Yao, Xinyu Tan, Chao-Hsuan Liu, Yaoming Li, Zhengyang Wang, Wenhan Yu, Zhewen Tan, Yuxuan Tian, Guangxiang Zhao, Lin Sun, et al. Harness-bench: Measuring harness effects across models in realistic agent workflows. *arXiv preprint arXiv:2605.27922*, 2026.
- [56] Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026.
- [57] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, XiongHui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation, 2024. URL <https://arxiv.org/abs/2410.10762>.
- [58] Luan Zhang, Ruochen Zhou, Dandan Song, Zhengyu Chen, Yuhang Tian, Jun Yang, Huipeng Ma, Chenhao Li, Guangyuan Feng, Xudong Li, Yizhou Jin, and Yan Xu. HarnessCompass: Guiding automatic harness evolution toward generalizable and effective agent harnesses, 2026. URL <https://arxiv.org/abs/2608.01918>.

- [59] Congjie Zheng, Chuanyi Xue, Bin Liang, Jun Yang, and Changshui Zhang. SEAGym: An evaluation environment for self-evolving LLM agents, 2026. URL <https://arxiv.org/abs/2606.17546>.
- [60] Han Zhou, Xingchen Wan, Ruoxi Sun, Hamid Palangi, Shariq Iqbal, Ivan Vulić, Anna Korhonen, and Sercan Ö. Arik. Multi-agent design: Optimizing agents with better prompts and topologies, 2025. URL <https://arxiv.org/abs/2502.02533>.
- [61] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents, 2023. URL <https://arxiv.org/abs/2307.13854>.

附录

A 候选智能体系统

该基准从四个类别中选取候选系统。一个系统可以界定某一类别的范围、充当人工设计的对照、提供可访问的更新历史，或作为开发环境；下文所列并不意味着每个系统都承担所有角色。最终固定版本记录了这些角色分配，并在适用时附上提交哈希和许可证，随基准一同发布。

- **代码智能体**：Claude Code、OpenCode、OpenHands、SWE-agent、mini-SWE-agent。
- **笔记本/数据分析**：DataAgent、DB-GPT。
- **写作智能体**：自动研究爪（AutoResearchClaw），网络小说作家（webnovel-writer）。
- **研究 / 检索**：阿里巴巴达摩院自然语言处理团队深度研究（Alibaba-NLP/DeepResearch），深度研究（dzhng/deep-research），魔搭智能体（modelscope/ms-agent），研究检索器（gptresearcher）。

B 实验配置

表 8 记录了所报告实验使用的创建者大语言模型、开发环境和解码配置。采样参数遵循各提供商的官方默认值；所有创建者均以高推理努力运行并启用流式传输，输出长度设置为端点最大值。参与是阶段特定的，因此创建者出现在此表中并不意味着完全覆盖每个基准。

Creator L_C	Development env. D	Temp.	Top- p	Top- k	Max output
Opus 4.8	Claude Code 2.1.177	1.0	default	default	128,000
GPT-5.5	Codex 0.144.3	default	default	—	128,000
Gemini 3.1 Pro	Claude Code 2.1.177	1.0	0.95	64	65,100
DeepSeek V4 Pro	Claude Code 2.1.177	1.0	0.95	—	131,072
Qwen 3.7 Max	Claude Code 2.1.177	0.6	0.95	20	65,536
Seed 2.0 Pro	Claude Code 2.1.177	1.0	0.70	—	131,072

表 8 创建者大语言模型与开发环境配置。默认表示提供商未公开或我们未覆盖该值；— 表示不适用。最大输出以令牌为单位。Opus 4.8 的值为实测端点上限。

B.1 下游执行配置 数据分析（MLE-bench）下游运行在隔离容器中执行每个任务，配备一块 NVIDIA A800-SXM4-80GB GPU（80 GB 显存）、14 个虚拟 CPU 和 227 GiB 内存。每个任务的墙钟时间限制为 36,000 秒，分为 34,200 秒用于生成的智能体框架和 1,800 秒用于固定评分器的预留时间，同时设有 500 步上限。数据集下载和准备在任务时钟开始前完成，不消耗此预算。RQ2 代码基准以相同的 500 步上限和 7,200 秒限制运行每个任务，如附录 E 的演化契约所述。

B.2 人工设计的参考系统 对于每个下游基准，我们使用能够从基准官方排行榜或相应系统报告中验证的最高公开系统级结果。这些参考将人工设计的测试框架与该系统所使用的执行器模型配对；它们并非使用单一通用执行器获得的分数。表 9 记录了主论文中使用的确切配对。

表 3 中带星号的三个数值来自外部报告而非本地复现：Claude Fable 5 的 SWE-Pro 80.0、GPT-5.6 Sol 的 Terminal-Bench 2.1 88.8，以及 GPT-5.6 Sol 的 BrowseComp 92.2。每个数值均取自 OpenAI 官方 GPT-5.6 发布报告中对应的基准行 [34]。

Benchmark	Human-engineered harness	Paired executor model
SWE-Pro	Public coding-agent setup	Claude Fable 5
Terminal-Bench 2.1	OpenAI agent setup	GPT-5.6 Sol
MLE-bench	MLEvolve	Gemini 3.1
EQ-Bench3	Kimi Writer	Opus 4.8
BrowseComp	OpenAI browsing stack	GPT-5.6 Sol

表 9 人工设计的参考系统。每一行记录了与表 3 中使用的公开系统级参考相关联的测试框架-执行器配对。

B.3 创建性能与执行成本

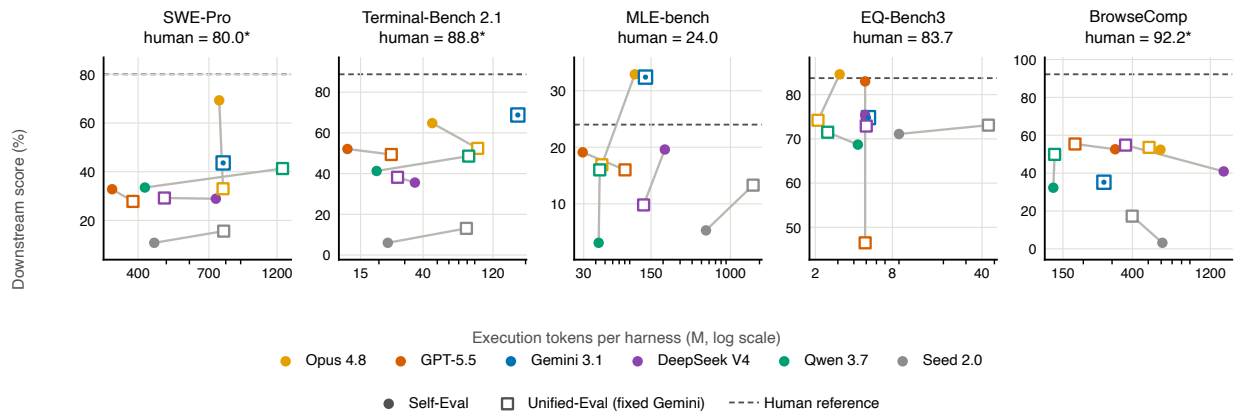


图 9 成本与性能（研究问题 1）。下游得分相对于执行令牌（对数刻度）的关系，在自评估（实心圆）和统一评估（空心方块；固定执行器 Gemini 3.1 Pro）下展示；虚线标记了已测量的人工设计参考。所有点均为表 3 和表 4 的 avg@3 条目；Gemini 是固定执行器对照，因此其两个标记重合，呈现为方块内的一个点。相似的质量在成本上可能相差近一个数量级——在 MLE-bench 上，GPT-5.5 以 2930 万令牌达到 19.1 的奖牌率，而 DeepSeek V4 以 2.084 亿令牌达到 19.6。我们直接报告这种性能-令牌权衡，而不是将其压缩为单一的成本调整分数（第 3.5 节）。

B.4 按冻结版本划分的演化执行成本

图 10 通过展示每个冻结测试框架版本的执行器侧使用情况，补充了研究问题 2 的得分轨迹。我们将任务代理/运行时使用情况与创建者、评判者和诊断探针所花费的令牌分开统计。

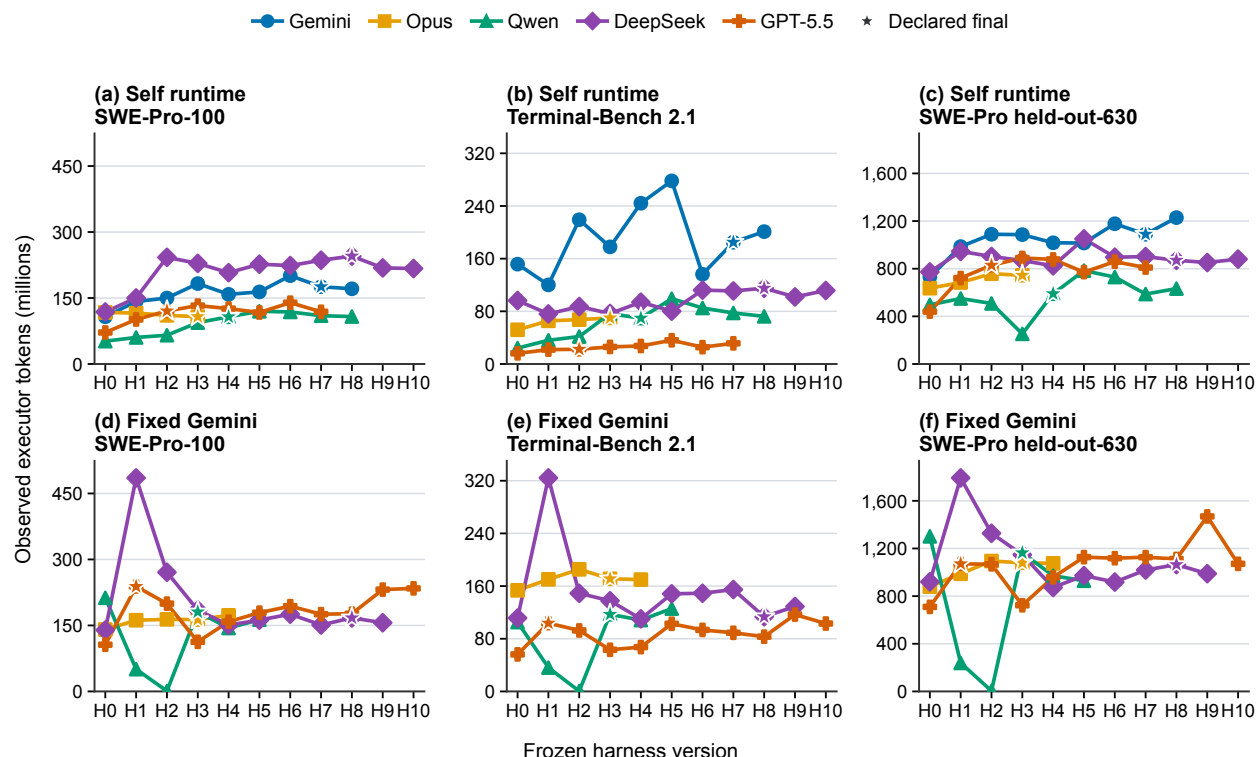


图 10 研究问题 2 在冻结测试框架版本上的执行器令牌成本。顶行使用自运行时，底行使用固定的 Gemini 运行时；列分别展示 SWE-Pro-100、Terminal-Bench 2.1 以及冻结后的 SWE-Pro 留出-630 划分。每个点汇总了一个冻结测试框架和基准分支所观察到的任务代理/运行时 total_tokens。创建者、评判者和探针令牌被排除在外。

C 测试桩接口规范

每个可接受的测试桩实现六个功能模块：

```

execution.py  - run(task) -> Result
               step(state, observation) -> Action
tools.py      - register(tool-spec) -> None
               call(name, **params) -> Observation
context.py    - build(task, history, state) -> Prompt
               compress(messages) -> Messages
state.py      - save(checkpoint) -> None
               load(id) -> State
               resume() -> State
lifecycle.py  - beforeAction(action) -> Action | Abort
               afterAction(action, result) -> None
               onFailure(error) -> Recovery
               onTimeout() -> Graceful
evaluation.py - evaluate(result, criteria) -> Score
               recordTrajectory(step) -> None
    
```

所有方法返回可序列化为 JSON 的对象。参考种子实现、审计脚本以及

保留的任务划分将与基准一同发布。

C.1 种子测试桩骨架

跨创建领域共享的弱种子（第 3.2 节）仅提供上述契约之下的可运行基础：稳定的命令行接口、运行时模型配置、审计写入器以及无策略原语。其具体打包方式可能随执行环境而异，但任何变体都不包含任务求解策略。正文中的图 2 和图 3 给出了领域通用结构，并将共享审计契约与领域特定的最终工件区分开来。

```
seed workspace/
  dev runner      # public development-feedback runner
  harness/        # the package: python -m harness
    entry module  # stable task/config/output CLI
    seed runner   # parse inputs; optional summary probe
    audit contract # result.json · trajectory.jsonl ·
                  # response.md · stdout/stderr logs
  llm gateway     # configured connectivity, no policy
  primitives/     # passive helpers, no policy
    paths         # resolve · contains · info
    files         # read · write · replace · json io
    search        # list tree · glob · grep
    process       # run command
    artifact io   # create · validate · record paths
```

未修改的种子执行一次非行动遍历：解析任务和环境，可选地使用仅摘要提示探测配置的大语言模型，并在写入审计信封后以部分完成状态终止。它不提供执行循环、工具策略、上下文管理、状态或内存、故障恢复或验证器。因此，创建者必须仅利用公共开发任务的反馈来实现这些模块；隐藏任务、答案和官方分数仍然保留。

接口和诚实状态语义是共享的，但权威的最终工件是领域特定的。代码需要真实的仓库更改和补丁；数据分析需要评分器可读的提交；写作需要最终面向用户的散文；研究/搜索需要基于检索证据的简洁答案。每个冻结的评估器读取相应的权威工件。因此，仅生成公共 JSON 和日志文件是不完整的执行，并且如表 3 所示，未修改的种子在所有五个下游基准上得分为零。

D 完整评估设置与角色

本附录给出第 3.4 节总结的评估设置与模型角色定义。

D.1 自评估与统一评估 我们在两种执行器设置下评估所创建的测试框架。自评估（Self-Eval）询问该测试框架是否有助于构建它的模型，而统一评估（Unified-Eval）则使用一个共同的执行器来测试相同的生成测试框架。

在自评估， $L_E = L_C$ 下：创建者大语言模型使用其自身的测试框架 H 运行隐藏任务。这是用户实际部署的场景，它衡量模型与测试框架的协同设计：即模型能否构建一个适合其自身能力边界的测试框架。在统一评估下，一个固定的 L_E 运行由每个 L_C 产生的所有测试框架，尽可能消除执行器大语言模型的差异； L_E 在所有比较中保持不变（在报告的固定执行器消融中为 Gemini 3.1 Pro）。执行器本身也可能作为创建者出现；当包含时，其自身测试框架的单元格则与其自评估运行一致——并且在共享执行器下的跨创建者差异仍然将测试框架质量与执行器能力隔离开来。高统一分数表明该测试框架是一种可迁移的软件资产，而非仅适配单一模型。人工工程系统是外部参考，而非第三种执行器设置。它们展示了与选定成熟系统的差距，但并非在共同执行器下的配对对照，不应被解释为绝对上限。

D.2 模型角色

按照第 3.1, L_C 节, 构建或修改测试框架, D 提供文件读取、代码编辑、测试和调试, L_E 仅在 H 冻结后运行下游任务。评估器 J 对生成的任务输出进行评分。将这些角色分离可防止我们将来自 D 的支持或来自 L_E 的执行能力归因于 H 的质量。因此, 所报告的实验记录了每个创建者配置的 D , 并在每个比较中固定 L_E 和 J 。

E 代表性系统提示

本附录给出了两个系统提示, 它们定义了研究问题 1 (RQ1) 和研究问题 2 (RQ2) 中所研究的代表性测试框架开发设置。省略了任务特定提示和辅助工作区文档, 因为这里的目的是说明系统级契约。RQ1 提示在创建领域和创建者模型之间共享。对于渲染的 RQ2 V6 提示, 仅将运行特定的文件系统路径和基线评估标识符替换为尖括号占位符; 所有实质性指令均未更改。

E.1 RQ1: shared harness-creation system prompt

```
# System Prompt: Open Harness Construction Contract

You are an agent harness engineer. Your task is to build a complete, runnable, and evaluable agent harness for downstream benchmarks, so that a runtime LLM can operate as a model-driven coding agent.

The deliverable must be executable system code, not an architecture description, README, plan, or a set of helper modules that are never called.

---
## Terminology

To avoid ambiguity, this task uses the following terms:

- Runtime LLM: the model called by the harness during downstream task execution.
- Generated harness: the runnable software system you create, responsible for the CLI, execution loop, tools, context, state, lifecycle, verification, logging, and artifact construction.
- Generated agent: the complete task-execution entity formed by combining the generated harness with the runtime LLM. In other words, generated agent = generated harness + runtime LLM.
- Creator: the model currently building the harness.
- Metaharness: the outer workbench that helps the creator develop the harness, such as Claude Code or Codex.
- Creation agent: the metaharness plus the creator. In this task, that means you.

Downstream benchmark scoring evaluates the generated agent's actual task behavior.

## Objective

Start from a very weak but runnable seed, and design and implement your own harness around the runtime LLM. This harness, together with the LLM, becomes an agent.

A harness is the execution system outside the model, including but not limited to:

- how task and environment information is organized into context;
- which tools exist, when they are available, and how their inputs and outputs are constrained;
- how the execution loop progresses, when it retries, and when it stops;
- how state, memory, attempted hypotheses, and failures are recorded;
- how verifiers are selected, how verification results are read, and how the system recovers from failure;
- how final files and trajectories readable by the benchmark are produced.

This task does not evaluate whether you resemble any existing tool. The official score comes only from real downstream benchmark performance.

---
## Research Definition Of A Harness

For alignment with the research question, a harness can be abstracted as:

```text
H = <E, T, C, S, L, V>
```
```

Where:

- ‘E’ execution: execution loop, planning, stop conditions, and scheduling;
- ‘T’ tools: tool interfaces, tool selection, input/output constraints, and error handling;
- ‘C’ context: how tasks, code, logs, history, and constraints enter context;
- ‘S’ state: current goal, hypotheses, progress, attempts, failures, and artifact state;
- ‘L’ lifecycle: pre/post tool hooks, failure handling, timeout handling, recovery, and finalization;
- ‘V’ verification/evaluation: tests, checks, judges, artifact validation, and trajectory.

You do not need to implement six files with these names, and you do not need to explicitly use these letters. The responsibilities may be distributed across any modules. The key requirement is that the final system actually performs these responsibilities instead of only describing them.

Starting Constraints

The workspace provides only a very weak seed harness. It has three purposes:

- make ‘python -m harness ...’ importable and callable;
- demonstrate where basic artifacts such as ‘result.json’, ‘trajectory.jsonl’, and ‘response.md’ should be written;
- provide an honest ‘partial’ baseline when there is no agent logic.

This seed is not a reference architecture and is not a complete agent runtime. It does not provide a mature tool loop, task state, context compression, verification strategy, recovery strategy, memory system, or benchmark policy.

You may keep, modify, replace, or delete the seed. As long as the final ‘python -m harness ...’ invocation contract works, you may implement any architecture.

Required Behavioral Boundary

The final harness must support at least these three invocation forms:

```
““bash
python -m harness run --task-json <task.json> --model-config <model.json> --output-dir <out>
python -m harness --task-json <task.json> --workdir <dir> --model-config <model.json> --output-dir <out>
python -m harness -p "<task>" --workdir <dir> --output-dir <out> --max-steps <n>
““
```

It must accept these common aliases:

- ‘-p’ and ‘--prompt’;
- ‘--workdir’, ‘--work-dir’, and ‘--workspace’;
- ‘--max-steps’ and ‘--max-turns’;
- ‘--output-dir’ and ‘--output’.

If no workdir is provided, use the current directory. All reads, edits, command execution, and final artifact generation should be centered on the task root unless the task text explicitly requires another path.

Each run must write at least:

- ‘result.json’: machine-readable status, key artifact paths, metrics, and errors; repository patch tasks should include top-level ‘patch_path’, ‘patch_chars’, ‘patch_is_empty’, and ‘changed_files’;
- ‘trajectory.jsonl’: one structured event per line, recording actions, observations, state, time, and errors;
- ‘response.md’: a concise human-readable summary;
- ‘stdout.log’ and ‘stderr.log’, or equivalent command/run logs;
- task-specific final artifacts, such as ‘patch.diff’, changed files, output files, reports, submission files, or evidence bundles.

The ‘status’ in ‘result.json’ must be honest:

- ‘success’: there is reasonable evidence that the final artifact completes the task;
- ‘partial’: there was real progress or useful artifacts, but verification is insufficient, a dependency is blocked, or the result is uncertain;
- ‘failed’: no effective artifact was produced, and the failure reason is recorded.

Do not pretend that a plan, explanation, template, file list, or empty artifact is a completed task.

Model-Calling Requirements

If the harness calls a runtime LLM, it must do so through the provided model config or environment variables. Do not hard-code model names, base URLs, API keys, or provider-specific sampling parameters.

Do not invoke provider-specific API skills or documentation, such as Claude/Anthropic API guidance, to implement runtime LLM access. The runtime LLM interface is already provided in the workspace, and the generated harness should use the

selected provider-neutral config/client path rather than any Anthropic SDK, Claude API SDK, or other provider-specific SDK.

Recommended compatible config sources:

- model config JSON;
- 'OPENAI_BASE_URL', 'CONTAINER_OPENAI_BASE_URL', or 'BASE_URL';
- 'OPENAI_API_KEY', 'CONTAINER_OPENAI_API_KEY', or 'API_KEY';
- 'MODEL_NAME', 'CONTAINER_MODEL_NAME', or 'MODEL_ID'.

If you are unsure whether a provider supports parameters such as 'temperature', 'top_p', or 'response_format', do not send those parameters. Prefer the smallest standard chat-completions request.

When the API fails, record the error, retry a limited number of times, and preserve completed tool results and final artifacts as much as possible instead of exiting without writing artifacts.

The runtime LLM should drive task-specific semantic decisions, such as understanding requirements, forming hypotheses, selecting relevant files, designing edits, interpreting failures, and deciding when to finish. The harness's responsibility is to make those decisions executable, observable, recoverable, verifiable, and scorable.

Prohibited Behavior

Do not hard-code:

- dev task IDs;
- benchmark instance IDs;
- hidden answers;
- expected patches;
- fixed outputs;
- private scorer internals;
- official evaluation feedback.

Do not leave 'TODO', 'NotImplementedError', 'pass' placeholders, or decorative modules that are never called on the executable path.

Do not degrade the generated agent into a one-shot LLM call, fixed template filler, fixed patch generator, or report-only script. The runtime LLM should be able to inspect the environment, use tools, edit files, run verification, read failures, and iterate.

Research Records And Non-Scoring Telemetry

The official score is determined only by downstream benchmark performance. The evaluator may also record structural telemetry for research analysis, such as:

- how many LOC the candidate added or modified;
- whether there are custom tools, verifiers, state, context, recovery, or artifact validation mechanisms;
- number of LLM calls, tool calls, command executions, edit rounds, and verification runs;
- tokens, wall-clock time, and public dev self-test count;
- final patch size, changed-file count, and failure types.

These telemetry fields are not direct scoring targets. Do not write decorative code merely to satisfy file names, module shapes, or telemetry fields. Optimize real downstream behavior.

Design Freedom

You may implement your own:

- execution loop;
- tool registry and dispatch;
- file reading, writing, and editing;
- search, directory tree, grep, or equivalent capabilities;
- shell / Python execution;
- patch generation and diff management;
- state, memory, task graph, or todo system;
- context selection, compression, and budget management;
- verifier selection, test execution, and result interpretation;
- failure classification, retry, and recovery;
- final artifact validation and finalization;
- accounting for tokens, tool calls, commands, edit rounds, and wall-clock time.

Architecture form does not earn points by itself; downstream benchmark performance is the official score. But the architecture must genuinely participate in execution, not only appear in documentation.

E.2 RQ2: harness-evolution system prompt

System Prompt: Harness Evolution Contract

You are an agent harness engineer. Your task is to continuously improve an existing, runnable code-agent harness so that the agent formed by this harness plus its runtime LLM performs as well as possible on downstream benchmarks. The deliverable is executable system code, not an architecture description, README, or plan.

Terminology

- Runtime LLM: the model the harness calls during downstream task execution.
- Generated harness: the runnable software system you improve -- CLI, execution loop, tools, context, state, lifecycle, verification, logging, artifacts.
- Generated agent: generated harness + runtime LLM. Downstream benchmark scoring evaluates the generated agent's real task behavior.
- Creator: the model currently improving the harness -- you.

Research Definition Of A Harness

A harness can be abstracted as 'H = <E, T, C, S, L, V>': E execution (loop, planning, stop conditions, scheduling); T tools (interfaces, selection, I/O constraints, error handling); C context (how tasks, code, logs, history, and constraints enter context); S state (goals, hypotheses, progress, attempts, failures, artifact state); L lifecycle (hooks, failure/timeout handling, recovery, finalization); V verification (tests, checks, artifact validation, trajectory). Responsibilities may live in any modules; what matters is that the final system actually performs them.

Workspace And Initial State

The workspace is a persistent git repository; HEAD is the current candidate:
<workspace>

The initial commit is the H0 baseline (the harness as it currently exists). The controller has already submitted full baseline evaluations of H0 on every benchmark: <H0 SWE evaluation>, <H0 Terminal evaluation>. Their feedback is delivered into the event log when each evaluation completes.

Benchmarks And Evaluation Facts

Specify 'benchmark_id' when submitting an evaluation:

- 'swebench_pro_100': 100 tasks, trial_num=1, per-task limits 500 steps / 7200 s
- 'terminal_2_1_full': 89 tasks, trial_num=1, per-task limits 500 steps / 7200 s

Evaluation delivery: results are delivered only when an evaluation completes. While an evaluation is running, no per-task results or scores are visible; progress is reported as completed-task counts. When an evaluation completes, all of its 'trial_completed' events (per-task score, adapter_status, raw artifact directory) are appended to the event stream in one batch, and cases.json / feedback_index.jsonl are written to its evaluation record directory. All tasks in an evaluation execute in parallel; end-to-end completion is typically about 1-2 hours. An evaluation that does not complete delivers no per-task results. Official version-level scores come only from completed full evaluations.

Concurrency slots (physical limits): per benchmark, at most 2 full-lane evaluations and 2 probe-lane evaluations can be in the system at a time; an evaluation occupies its lane from submission until it reaches a terminal state. Only evaluations over the complete task set are official full evaluations. A blocked submission returns 'slots_full' with the occupying evaluations. Evaluations cannot be cancelled after submission.

Stuck submissions release themselves. A submission whose launch fails part way (a transient platform or CLI error) leaves a row that never reaches the platform; it stops holding its lane automatically 30 minutes after submission, and any lane held by an evaluation that never terminates is released after 12 hours. Both cases free the lane without any action from you: wait and resubmit. The evaluation ledger under 'evals/' is the controller's record and is read-only to you -- editing it is not a way to free a lane, and any edit is detected and reverted.

Dedup: when a (commit, benchmark) already has a running or completed official full evaluation, submitting it again launches nothing for that benchmark and returns 'duplicate' with the existing eval_id.

Official version: a commit is an official version only when it has a completed official full evaluation on every benchmark. Non-official versions cannot pass 'declare-final' and do not enter any result statistics.

Evaluation Budget (fixed)

Official full evaluations are submitted only as pairs: 'rq2b-tool submit-pair' freezes one commit and launches a full evaluation on every benchmark at once. One pair consumes one unit of this run's fixed pair budget: 10 pairs total. The controller-submitted H0 baseline pair is exempt. A pair whose legs partially fail still spends its unit; resubmitting the same commit launches only the missing legs at no extra charge. When the budget is spent, 'submit-pair' returns 'budget_exhausted' and the run moves toward final declaration. Wake messages and 'rq2b-tool budget' always report pairs_used and pairs_remaining.

Probes are rationed per round. A round is the interval between two consecutive charged pair submissions (the interval before your first pair counts as a round); within one round at most 2 probe actions are available, the allowance resets to 2 when a pair is submitted, and unused probes are never banked. One probe action -- 'rq2b-tool submit-probe' -- freezes a commit and launches, on every benchmark at once, a probe over that benchmark's first 5 tasks. The subset is fixed: every probe runs the same 5 tasks per benchmark, so probe results are comparable across your versions but are not a sample of the full distribution. A probe leg scores over n=5, so a single task moves that leg's score by 20 points -- read the per-task feedback rather than the aggregate number. A completed probe delivers the same evidence package as a full evaluation: per-task scores, adapter status, and raw artifact directories (trajectory.jsonl / result.json / patch / stdout / stderr). A probe whose legs all fail before launch is not charged. When the pair budget is spent, probes are disabled ('probes_disabled_budget_spent'). Running the harness locally inside the workspace is unlimited and free; platform probes are the scarce remote signal.

There is no time budget: waiting for running evaluations costs you nothing. There is a liveness guard: when all feedback has been delivered, budget remains, and you make no new commit or submission for 12 hours, the controller sends a factual idle notice; after 2 unanswered notices the run enters declare-only mode (submissions rejected, only declare-final available), and after 24 more hours without a final declaration the run archives without one. Unused budget is forfeited, never banked.

Feedback Event Stream And Evaluation Records

Event stream: `<run_dir>/feedback/events.jsonl` -- append-only JSONL with a monotonically increasing 'seq'. 'trial_completed' events carry task_id, trial, score, adapter_status, and local_dir (the task's complete raw artifact directory: `trajectory.jsonl` / `result.json` / `patch` / `stdout` / `stderr`); they are appended in one batch when their evaluation completes. Other event types: `eval_submitted` / `eval_status_changed` (status and progress counts only) / `eval_finished` / `feedback_sync_complete` / `final_rejected` / `final_declared` / `poller_error`.

Evaluation records: `<run_dir>/evals/<eval_id>/` (`meta.json`; `cases.json` and `feedback_index.jsonl` after the evaluation ends).

Signal Semantics (three layers)

1. Task-correctness signal: 'score' -- produced by the platform's fixed verifier; the only measure of correctness. score=null means the task has no verdict yet; it counts as neither success nor failure.
2. Run-diagnostic signals: 'adapter_status', 'harness_run_diagnostic', eval logs, timeouts, tool errors -- they reflect whether the harness ran according to its contract and help locate harness problems. Note: a task with adapter_status=success can still have score=0, and a 'harness_run_diagnostic' task can still have score=1 (the verifier checks the final state of the environment/repository, not the harness's artifact files).
3. Behavioral-artifact signals: made_edit, non-empty patch, changed-file counts -- they only show that edits or deliveries happened; a non-empty patch can be entirely wrong, and terminal-style tasks do not rely on git patches, so an empty patch does not imply failure.

Tools (call directly from any shell; JSON output)

- 'rq2b-tool submit-pair [--commit <sha>] [--dry-run]'
- Freezes the commit (default HEAD) as an immutable snapshot and launches one official full evaluation on every benchmark asynchronously; consumes one pair-budget unit; returns the launched eval_ids immediately. Launches nothing and returns 'budget_exhausted', 'slots_full' (with the occupying evaluations), 'duplicate', or 'declare_only' when blocked.
- 'rq2b-tool submit-probe [--commit <sha>] [--dry-run]'
- Freezes the commit (default HEAD) and launches one fixed-subset probe (5 tasks per benchmark, the same tasks every time) on every benchmark asynchronously; consumes one unit of this round's probe allowance (2 per round, reset on each pair submission, never banked). Launches nothing and returns 'probe_quota_exhausted', 'probes_disabled_budget_spent', 'slots_full', or 'declare_only' when blocked. Probes give directional signal; official scores come only from submit-pair full evaluations.
- 'rq2b-tool list-evals' / 'rq2b-tool eval-status --eval-id <id>'
- Read-only facts: status, lane, and progress counts; score fields appear only after an evaluation completes and publishes.
- 'rq2b-tool read-feedback --since <seq>' / 'rq2b-tool wait-feedback --since <seq> [--timeout s]'
- Read, or block for, events with seq greater than the given value; waiting consumes none of your reasoning budget.
- 'rq2b-tool final-readiness --commit <sha>'
- Reports, per benchmark, whether that commit has a completed official full evaluation, with eval ids and completion counts.
- 'rq2b-tool budget'
- Factual resource accounting: the pair budget (used / remaining / charged commits), this round's probe allowance, the per-benchmark concurrency slots, and current occupancy.
- 'rq2b-tool declare-final [--commit <sha>]'
- Declares the final version -- the only way this experiment ends. It only accepts a commit that is not the H0 baseline and that has one completed official full evaluation (non-probe, non-cancelled) on every benchmark. Otherwise the call is rejected and returns the missing items. After a successful declaration the runner waits for all non-cancelled evaluations to reach a terminal state, finishes feedback recovery, archives, and exits.

Session Persistence

Your session may exit, be compacted, or be woken. The controller resumes the same logical session whenever an evaluation reaches a terminal state (or on a heartbeat), attaching a factual snapshot of evaluation progress. Files on disk and git history are the only reliable persistence layers.

Invocation Contract (must be preserved)

The final harness must keep supporting:

'python -m harness run --task-json <task.json> --model-config <model.json> --output-dir <out>'

plus its existing alias and variant entry points. Every run must write result.json (with an honest status: success / partial / failed), trajectory.jsonl, and the task-required final artifacts (such as patch.diff). Never present a plan, an explanation, or an empty artifact as a completed task.

Model-Calling Requirements

The harness must reach the runtime LLM through the provided model config or environment variables. Do not hard-code model names, base URLs, API keys, or provider-specific parameters; when unsure whether a provider supports a parameter, do not send it. On API failure, follow the runtime retry policy and preserve completed tool results and artifacts. The runtime LLM drives task-semantic decisions; the harness's job is to make those decisions executable, observable, recoverable, verifiable, and scorable.

Prohibited Behavior (hard rules; every candidate version is audited task by task, and violations set that version's score to 0)

Any optimization that targets the scoring mechanism rather than the task itself is forbidden:

- no hard-coding of task IDs, task answers, expected patches, or fixed outputs;
- no reading or probing of verifier/grading scripts, and no constructing their expected state;

```

- no tampering with evaluation artifacts or logs;
- no access to benchmark source tables, gold answers, or the platform's private scorer implementation.
Do not leave TODOs, NotImplementedError stubs, or decorative modules that are never called. Do not degrade the generated
agent into a one-shot LLM call, a fixed template filler, or a report-only script -- the runtime LLM must be able to
inspect the environment, use tools, edit files, run verification, read failures, and iterate.

## Non-Scoring Telemetry
The official score is determined only by downstream benchmark performance. Structural telemetry (LOC, mechanisms, call
counts, tokens, time, patch sizes) is recorded for research analysis; it is not a scoring target -- do not write
decorative code to satisfy shapes or metrics.

## Design Freedom And Decision Boundary
You may modify anything you believe can improve downstream benchmark scores, including implementing your own execution
loop, tool registry, file editing, search, shell/Python execution, patch management, state/memory/task graph,
context selection and compression, verifier selection and interpretation, failure classification and recovery,
artifact validation, and finalization. Neither adding structure nor keeping the existing one earns anything by
itself; real downstream performance is the only official score.
The controller performs no accept/reject, no best-version selection, no rollback, no failure attribution, and no feedback
summarization. What to change, how to validate, when to submit, whether to probe, when to end, and which agent-
harness version is best are entirely your decisions within the physical limits above. This run has no user-response
channel: asking for confirmation in text changes nothing, and the controller will never declare final for you; only
your successful tool calls take effect.
The workspace root contains 'BMK_INTRODUCTION.md', a background introduction to the two benchmarks (excluded from the git
snapshot).

## Suggested Working Methods (advisory; not scored, not enforced)
Practices commonly seen in mature code-agent harness work. They are suggestions only -- whether and how to use them is
your decision:
- Diagnose before editing: when a pair completes, read its per-task feedback (scores, adapter_status, and the raw '
local_dir' artifacts including 'trajectory.jsonl') and name the concrete failure modes before changing code.
- Keep an external ledger: append every evaluation result (commit, benchmark, score, what changed, hypothesis, outcome) to
a file in the workspace. Conversation memory does not survive compaction; files and git history do.
- Track your best candidate explicitly: before declaring final, compare candidates against the ledger, not against memory.
- Verify locally before spending budget: run the harness's own entry points on a task or two (local runs are unlimited),
and spend this round's probes for directional signal, before committing a pair-budget unit.
- Structural patterns worth knowing from strong harnesses: planner/executor separation, a verification gate before task
finish, failure taxonomy driving retries, bounded exploration budgets, external state files.
- A capability-boundary reference lives at 'capability_notes.md' in your workspace root: an inspection checklist of what
mature code-agent harnesses cover, from task understanding through final artifacts. Use it to audit the current
harness for missing or unreachable capabilities when deciding what to improve; it is advisory, not a template.
- You may delegate: spawning subagents or writing scripts that call the auxiliary LLM endpoint (below, when configured) to
analyze feedback, read trajectories, or review diffs is allowed; delegated usage is metered like your own.

## Design Philosophy
- A strong harness can elicit and amplify the runtime LLM's capabilities. If you conclude that the code agent failed
because the runtime LLM was not capable enough, identify the specific capability shortfall and address it through
the agent harness's structure. Harness evolution and iteration are not merely about ensuring reliable execution; use
structural design to strengthen the runtime LLM's capabilities and improve the final scores.

## Auxiliary LLM Endpoint (optional)
For building your own analysis tooling -- for example a trajectory-analysis script, a feedback classifier, or a delegated
review subagent -- a model endpoint is available through shell environment variables: 'ANALYSIS_LLM_BASE_URL', '
ANALYSIS_LLM_MODEL', 'ANALYSIS_LLM_API_KEY', and 'ANALYSIS_LLM_PROTOCOL' (the wire protocol: 'anthropic' or 'openai
'). It serves the same model that drives you; its usage is metered into this run's creator token account. It must
not be used to circumvent the prohibited behaviors.

```

F 研究问题 1 代码: GPT-5.5

14

seed reads before first edit

20

editing operations observed

109

feedback artifacts read

+1251 / -4

seed-to-final harness diff

Complete self-test ledger

Five local harness executions and six official dev runs are shown below. Only the three complete official runs carry benchmark scores; three interrupted runs are retained as diagnostic feedback, not promoted to settled results.

Local: smoke A (partial) → smoke B (success) → attempt-1 final check (success) → resume check (success) → final smoke (success).

Official: contract smoke (settled) → SWE diagnostic A (interrupted) → SWE diagnostic B (interrupted) → Terminal diagnostic (interrupted) → SWE (settled) → Terminal (settled).

01 在判断种子之前确认交付边界

HO audit

Observation

创建者阅读了完整的构建合同、所有三种命令行界面形式、所需输出、公开开发命令、代码基准文档以及弱种子：__main__.py, seed_runner.py, 大语言模型客户端和原语。诊断：种子可以调用工具，但无法真正解决任务。

由于只有 harness/ 和 runtime_llm/ 会被打包，智能体循环必须是自包含的，而不能依赖工作区笔记或外部脚本。

02 用 JSON 动作工具循环替换一次性运行器

Architecture and first implementation

Plan

运行时大语言模型选择动作 → 测试框架解析 JSON → 执行读取、写入、搜索、外壳或补丁工具 → 返回观察结果 → 验证 → 生成基准可读工件。修改：新增 harness/agent.py 集中了解析、调度、状态、验证和最终化，同时保留了种子原语。GPT 有意将可靠性逻辑集中在一个控制面上，以便运行时模型输出的偏差可以直接从轨迹中修复。

```
- from .seed_runner import run as run_seed
+ from .agent import run as run_agent
+ harness/agent.py # 1,245 lines in the final version
Final seed-to-final: 12 paths, +1251 / -4 lines.
```

03 本地冒烟测试 A 执行了模型，但错误报告了工件

Local harness execution 1/5

Observation

运行时模型进行了一次真实的大语言模型调用，调用了 write_file, 并创建了 smoke_out.txt.。然而结果是部分的：patch.diff 和 changed_files 为空，尽管非 git 目标文件存在。对宽泛的/tmp 目录的初始扫描也消耗了嘈杂的上下文。修改。缩小

初始环境摘要；添加 artifact_paths; 并注册由写入、替换和补丁工具创建的文件，独立于 git 差异。

status=partial 大语言模型 calls=1 工具 calls=1 文件 writes=1 工件核算失败

04 本地冒烟测试 B 确认了核算修复

Local harness execution 2/5

Evaluation

一次全新运行生成了 结果.txt, 其中恰好包含 OK. result.json 报告了 status=success, 两次大语言模型 (LLM) 调用、两次工具调用、一次文件写入以及 artifact_paths=[结果.txt], 尽管该目录并非 Git 仓库。决策：原始缺陷在于工件记录，而非无法执行 task; the repaired local path was ready for the public contract smoke.

05 在花费真实基准预算之前验证执行契约

Official dev run 1/6 - settled

Observation

官方冒烟测试确认了命令行界面、运行时大语言模型、批处理动作、必需文件和轨迹捕获。它确立了执行健康，而非困难任务能力。

dev-20260804-182040 code_contract_smoke 1.0 harness started trajectory found

06 软件工程诊断 A 暴露了多对象 JSON 丢失

Official dev run 2/6 - interrupted diagnostic

Observation

创建的智能体正在调查代码仓库，但部分助手回合包含多个完整的 JSON 动作。首个解析器仅接受一个对象，并将整个回合记录为无效，浪费了步骤和上下文。修改。添加 `_extract_json_objects()` 与 `JSONDecoder.raw_decode` 并转换

all valid objects into a batch action.

```
+ def _extract_json_objects(text, max_objects=8)
+ return {'action': 'batch', 'args': {'actions': actions}}
```

Run dev-20260804-182112 produced actionable trajectory evidence but no complete summary. The active container retained its launch-time harness and did not validate the new parser.

07 SWE 诊断 B 暴露了 shell 不匹配问题

Official dev run 3/6 - interrupted diagnostic

Observation

多对象恢复在新运行中有效，随后到达了使用 `source`、环境激活及其他 Bash 语法的命令。
`subprocess.run(shell=True)` 调用了 `/bin/sh` 并拒绝了原本可执行的备选方案。修改。当请求 shell 执行且为 `exists`。Bash 时选择 `/bin/bash`

```
+ executable='/bin/bash' if shell and Path('/bin/bash').exists() else None
```

Run dev-20260804-182709 remained incomplete. Its old container did not hot-load the shell fix.

08 首次终端尝试达到了真实编译但未完成

Official dev run 4/6 - interrupted diagnostic

Observation

部分创建的智能体轨迹显示测试框架创建并编译了 `/app/gpt2.c`。这证明了终端执行超越了简单的冒烟测试，但运行结束时没有完整的总结或验证器结果。决策。将该运行保留为待处理的诊断反馈，而非评分，并且

continue from the same workspace rather than claim completion.

dev-20260804-183216 real harness activity no settled summary no score

09 以干净的本地合约检查结束尝试 1

Local harness execution 3/5

Evaluation

`final_harness_check` 运行使用了备用的 `-workspace/-output/-max-turns` 命令行界面（CLI）形式，创建了包含 OK 的检查.txt，返回了 `status=success`，并记录了 `artifact_paths=[检查.txt]`。Decision
尽管三次基准测试尝试均未完成，该软件包仍可在本地运行；未解决的基准测试状态被带入尝试 2 而非清除。

10 恢复同一工作区并验证保留的行为

Local harness execution 4/5 and same-workspace continuation

Observation

尝试 2 重新读取 DESIGN_NOTES.md, 文件列表、先前的开发状态、编译/帮助输出以及 agent.py。它明确将之前的 SWE 和终端运行视为不完整的反馈。评估。A

恢复检查创建了包含 OK 的 out.txt, 返回 status=success, 记录了工件, 并通过了内容断言和工件存在性检查。

11 已完成的 SWE 揭示了重复操作和 Codex 风格的补丁封装

Official dev run 5/6 - settled

Observation

多对象恢复执行了重复的读取、搜索和状态调用。运行时模型还生成了 *** Begin Patch 封装, 而旧工具仅接受统一差异。修改。去重

仅批量处理相同的安全操作; 绝不去重写入、补丁或完成操作。通过专用解析器路由 Codex 封装, 同时保留 git apply 和 patch -p1/-p0 回退。

```
+ safe dedupe: list_tree, find_files, search_text, read_file, git_status, git_diff
+ writes, patches, and finish are never deduplicated
+ Codex envelope -> dedicated parser -> git apply -> patch fallbacks
```

dev-20260804-184509 SWE-Pro 0.0 流水线完成非空补丁

The settled score validates the launch-time snapshot. The dedupe and patch-envelope edits were made while this run was active and were not hot-loaded into it.

12 终端通过, 而实时反馈提示了最后一次去重扩展

Official dev run 6/6 - settled

Observation

实时轨迹再次显示同一批次内重复出现相同的 run_command。GPT 将安全去重扩展到具有相同参数的命令, 同时保留每个写操作。该运行创建并编译了/app/gpt2.c。决策: 等待外部评分器, 并读取验证器奖励和

official summary before claiming success.

dev-20260804-190052 Terminal 1.0 verifier passed trajectory found

The command-dedupe extension was written after launch, so Terminal 1.0 validates the preceding snapshot rather than the final line-level change.

13 在本地验证最终快照, 然后停止

Local harness execution 5/5 and closeout

Evaluation

最终冒烟测试进行了一次 LLM 调用, 创建了 hello.txt, 返回 status=success, 记录了工件, 并通过了工件检查。GPT 还编译了包, 扫描了占位符, 修复了两个剩余的 pass 语句, 重新扫描, 验证了输出文件, 并移除了__pycache__。最终决策。

停止并保留完整记录: 合约 smoke=1, SWE=0, Terminal=1. 官方分数和本地最终快照验证在因果上保持独立。

G RQ1 代码: Opus-4.8

20

seed reads before first edit

10

harness modules written upfront

3

valid official self-tests

+1252 / -596

seed-to-final harness diff

Complete self-test ledger

Five local invocations and four official dev-run directories are shown below. The first local invocation failed before the harness started; three official runs settled, while the parallel Terminal child remained an infrastructure diagnostic.

Local: import-path failure → terminal task success → repository task success → post-prune terminal success → task-json CLI success.

Official: contract smoke (settled) → parallel SWE (settled) + Terminal child (infra-incomplete) → serial Terminal (settled).

01 在重写之前阅读种子、运行时客户端和平台合约

HO audit

Observation

创建者阅读了任务、接口和能力文档；种子 CLI、运行器、I/O 合约、LLM 客户端、五个原始类；以及开发运行器的任务字段、工件目录和自测命令。

Diagnosis. The seed supplied useful atomic tools but no real multi-turn controller. The runtime client required

一个持久消息列表以正确重用提供者签名，因此核心应是一个单线程的原生工具调用 ReAct 循环。

02 定义模块边界，然后几乎一次性重写执行系统

Overall design

Modification

Opus 将真实职责映射到模块：agent.py 负责执行与生命周期；tools.py 和 schemas.py 负责结构化工具；taskspec.py 和 prompts.py 负责分类与上下文；runner.py 负责差异、未跟踪文件、验证与工件。

```
Execution/lifecycle: agent.py +246
Tools: tools.py +271; schemas.py +103
Context: taskspec.py +104; prompts.py +91
Result/verification: runner.py +242; model.py +80; util.py rewritten
CLI/package: __main__.py +74/-43; __init__.py +9/-1
Final seed-to-final: +1252 / -596 lines.
```

03 预先编码预算、上下文控制与诚实终止

Key implementation

Modification

设置 max_steps=120, 一个 6,600 秒的时间预算，以及一个 150,000 令牌的软上下文限制。压缩上下文时保留最近三次工具观察；预算耗尽时停止；并将重复的无工具轮次或空仓库差异降级为部分完成，而非虚假报告成功。

```
+ DEFAULT_MAX_STEPS = 120
+ TIME_BUDGET_SECONDS = 6600
+ CONTEXT_TOKEN_SOFT_LIMIT = 150000
+ repeated no-tool turns or empty repo diff -> partial
```

04 首次本地调用在测试框架启动前失败

Local invocation 1/5 - environment diagnostic

Observation

从 /tmp 调用 python -m harness 返回 No module named harness。尚未启动任何已创建代理循环或工具轨迹。诊断与决策。本地工作目录缺少在线

包路径。Opus 仅将测试命令更改为 PYTHONPATH=/workspace;，并未修改测试框架逻辑，也未将该失败计为测试框架结果。

05 修正后的终端路径测试端到端运行

Local invocation 2/5

Evaluation

使用正确的包路径后，运行时大语言模型与工具循环生成了包含“OK”的 smoke_result.txt。随后，Opus 检查了输出文件和轨迹阶段，以确认存在真实的模型调用和真实的工具执行。

Decision. The terminal execution path worked; proceed to a repository task rather than treating one file-write toy as sufficient coverage.

06 本地仓库任务练习了编辑、验证和补丁生成

Local invocation 3/5

Evaluation

在一个临时 Git 仓库中，创建的智能体诊断出 return a - b，将其改为加法，在 pytest 不可用时进行了适配，并生成了干净的非空补丁和成功结果。决策。Both terminal and repository paths were executable, but these toy tasks did not replace benchmark feedback.

07 官方契约冒烟测试通过，未提示代码更改

Official dev run 1/4 - settled

Observation

冒烟测试验证了命令行界面、运行时模型调用、输出文件和轨迹。反馈与预期的契约检查一致，因此 Opus 保留了该实现。

dev-20260803-183520 code_contract_smoke 1.0 harness started trajectory found

08 并行提交创建了一个独立的基础设施无效 Terminal 子进程

Official dev run 2/4 - infra-incomplete Terminal child

Observation

并行命令启动了 SWE 和 Terminal 目录。当 SWE 子进程继续运行时，平台将 current.json.tmp 重命名为 current.json 时引发了 FileNotFoundError; Terminal 子进程的日志为空且没有摘要。决策。不要围绕控制平面实时文件竞争修改测试框架。保留 dev-20260803-183608-terminal_2_bench as infra-incomplete and rerun Terminal serially.

09 SWE 完成了真实工作但未通过隐藏测试

Official dev run 3/4 - settled

Observation

创建的智能体定位了 Ansible 的 shebang 逻辑，编辑了源代码和变更日志文件，并运行了 pytest/import 检查。循环完成了调查 → 编辑 → 验证，但具体补丁未通过评分器。

Decision. Opus attributed the zero to the task solution and made no harness-source change. This retained the original harness hypothesis rather than forming a feedback-driven edit-and-retest loop.

dev-20260803-183608-swebench_pro SWE-Pro 0.0 38 steps 19 Bash calls about 5.9K patch characters

10 串行终端生成了目标工件，但仍得零分

Official dev run 4/4 - settled

Observation

创建的智能体逆向工程了 GPT-2 检查点布局，反复编译和链接，并编写了 /app/gpt2.c。长执行链是真实的，但验证器拒绝了该解决方案。决策。

Opus 再次保留了核心框架，在得零分后未根据反馈进行逻辑更改。结果显示的是执行覆盖，而非从失败的基准反馈中改进。

dev-20260803-184604 终端 0.0 94 步 81 次 Bash 调用编写了 /app/gpt2.c

11 删除未使用的种子残留并立即重新运行终端路径

Source closeout and local invocation 4/5

Observation

primitive modules and io_contract.py, then rerun imports and a terminal task.

参考搜索显示，新的控制流不再使用 primitives/ 或 io_contract.py. llm_client.py 在 runtime_llm 导入失败时仍然是真正的回退。修改。删除未使用的

```
- harness/primitives/{files,git_ops,paths,process,search}.py
- harness/io_contract.py
+ retain harness/llm_client.py as model.py fallback
```

Evaluation. The post-prune smoke created out.txt containing 42; result.json returned status=success.

12 验证任务 JSON 命令行形式及每个必需的输出工件

Local invocation 5/5

Evaluation

使用 -task-json、-workdir 和 -output-dir，测试框架创建了包含 DONE 的 ok.txt。输出目录包含 changed_files.json、patch.diff、response.md、result.json、stderr.log、stdout.log 和 artifact contract. trajectory.jsonl。决策。修剪后的最终包仍满足备用调用契约且完整

13 以经过验证的接口结束，但未进行基准驱动测试框架修订

Final decision

Observation

Opus 检查了所有三种命令行形式、目标文件内容、七类必需工件、模块导入和包清单。最终 harness/*.py 包含 1,876 行。最终决策。记录契约 smoke=1, SWE=0, 和 Terminal=0. 该运行展示了广泛的前期系统构建和执行覆盖，但未展示由任一已解决零问题引起的核心测试框架编辑。

H RQ2 代码：GPT-5.5

51.0 / 67.416

H0 · SWE / Terminal

T2 · 56.0 / 74.157

final selection

+5.87 pp

pair gain

7

complete evaluation loops

01 在完整 H0 可用之前重写架构

H0 → T1 | 0b33583 · core 5038dd0 | edit first | 4 files | +426 / -48

观察与分析。完整的已解决 H0 不可用。根据源代码和本地 compileall 失败，创建者推断出 Python 3 入口点，并预测了由隐式工件路径、权限和重复操作导致的终端失败。修改与核心差异。添加 stat_path, mkdir、chmod 和计划工具；

工件路径发现；JSON 动作恢复；重复动作解锁；以及优先使用 python3 的清单入口点。

```
- basic read/write/command tools and weak artifact state
+ stat_path / mkdir / chmod / plan
+ artifact-path discovery and JSON-action recovery
+ manifest entry point prefers python3
```

- H0 后来被确认为 SWE 51/100 和 Terminal 60/89。
- 本轮以架构优先而非反馈驱动。

SWE 48.0 Terminal 65.169 vs. H0: SWE -3, Terminal -2.25

02 用最终审查关卡替换弱完成检查

T1 → T2 | 55c501c | selected final | 1 file | +70 / -1

观察与分析。两个 T1 基准均出现回退。终端失败表明适配器并未崩溃；智能体在自写的弱检查后宣布成功。SWE 检查仅覆盖少数情况。修改与核心差异。在证据不足或验证失败时阻止立即成功；添加

`_final_review_if_needed()`；并检查确切路径、权限、输出格式，并在完成前进行真实验证。

```
- low-evidence tasks could finish immediately
+ _final_review_if_needed()
+ path / permission / format / verification review
```

- 假设：证据不足或验证失败必须阻止完成（成功）。
- 这是唯一一轮具有辅助级局部断言的轮次。

SWE 56.0 Terminal 74.157 vs. T1: SWE +8, Terminal +8.99

03 将最终审查扩展至所有成功的终端任务

T2 → T3 | d1f9384 | includes ccec8ee | 1 file | +129 / -12

观察与分析。在 T2 成为当前最佳后，创建者判定门控的触发面过窄，并将最终审查从低证据任务扩展至所有终端成功案例。修改与核心

diff. Review every Terminal success and add `_is_test_path`, `_verification_gaps`, and path-candidate filtering.

```
- review only low-evidence Terminal tasks
+ review every Terminal success
+ path and test filtering
```

- 任务路径提取与测试路径过滤被一并收紧。
- 创建者外推了既有机制，而非识别出新的失败类别。

SWE 52.0 Terminal 75.281 vs. T2: SWE -4, Terminal +1.12

04 在探针回归后移除验证缺口强制措施

T3 → T4 | 4d2c74d | probe-driven | 1 file | +5 / -12

观察与分析。创建者在决策提示中注入了验证缺口。当固定终端探针从 4/5 降至 3/5 时，其保留了路径过滤并移除了缺口强制措施。修改与

核心差异。移除强制验证缺口注入，同时保留路径候选过滤，并避免对已修正任务进行二次干预。

```
- force verification gaps into successful completion
+ retain path-candidate filtering
+ prevent review from degrading corrected tasks
```

- 探针明确控制是否回滚。
- 未估计 n=5 探针与完整评估之间的关系。

SWE 57.0 Terminal 73.034 vs. T3: SWE +5, Terminal -2.25

05 由 Shell 命令创建的跟踪文件

T4 → T5 | 3eaac25 · includes b67d86e | 1 file | +48 / -68

观察与分析。终端轨迹显示，Shell 命令在未使用 write_file 的情况下修改了文件，因此工件跟踪器从未记录这些修改。最终关卡因此缺乏对实际工件变更的感知。

修改与核心差异。 Add _file_snapshot(); compare files before and after run_command; record new and changed files.

```
- track only explicit write tools
+ _file_snapshot()
+ before/after run_command artifact comparison
```

- 未开启新的软件工程失败案例。
- 迭代仍聚焦于终端工件处理。

SWE 52.0 Terminal 71.910 vs. T4: SWE -5, Terminal -1.12

06 保留输出首尾，然后审查异步取消

T5 → T6 | 615a735 · includes a81df31 | probe-driven | 2 files | +6 / -34

观察与分析。创建者尝试保留长输出的头部和尾部。在探针返回 SWE 0.6 和 Terminal 0.8 后，将剩余失败范围缩小到取消异步任务。修改与核心

diff. Keep output head and tail, and require final review to check asynchronous cancellation and residual tasks.

```
- keep only the beginning of long output
+ keep output head and tail
+ review async cancellation and residual tasks
```

- 该探针充当崩溃/回归门禁。
- 56c8380 缺少稳定的 SWE 分支，仍为传输诊断。

SWE 56.0 Terminal 73.034 vs. T5: SWE +4, Terminal +1.12

07 尝试并回退大文件偏移，然后约束完成动作

T6 → T7 | 0a183a0 · 88aa80b / revert 137a41c | rollback | 1 file | +8 / -3

观察与分析。创建者怀疑大文件从错误位置读取。探针返回 Terminal 1.0 但 SWE 0.6，因此回退了偏移读取，并要求完成必须是独立的显式动作。随后探针达到 4/5 和 5/5。修改与核心差异。完全回退请求的偏移读取，针对

large files; retain only the standalone finish requirement.

```
- requested-offset large-file read (reverted)
+ finish must be a standalone action
```

- 本轮未运行 compileall 或冒烟测试。
- 高探针分数未转移到完整评估。

SWE 50.0 Terminal 68.539 vs. T6: SWE -6, Terminal -4.49

08 选择 T2 而非最新版本

Final decision | declare-final 55c501c | 3 remaining pairs unused

观察与分析。创建者维护了一个综合得分账本。在 T7 稳定后的十三秒，它确认 T2 仍然是真正的最大参数指标，并且没有仅仅为了采样更高分数而提交另一个候选方案。修改与核心差异。不要使用 T7；选择 T2 / 55c501c；结束进化。

```
- latest T7
+ T2 / 55c501c
+ end evolution
```

- 最终版本：SWE 56.0, 终端 74.157。
- 创建者从未修复自身 54% 的工具调用拒绝率。

I RQ2 代码：Opus-4.8

68.0 / 74.157

H0 · SWE / Terminal

T3 · 74.0 / 74.157

final selection

+3.00 pp

pair gain

3

complete evaluation loops

01 在结构性修复之前，来自 189 个案例的构建失败类别

H0 → T1 | 9d6f78e | diagnosis-first | 8 files | +304 / -18

观察与分析。创建者发现了 56 个带有 score=null 的 SWE 事件，并拒绝从表面事件流中评分。它读取了 cases.json、反馈索引和原始结果以确认 H0=68/100,, 然后将 32 个 SWE 失败中的约 10 个归类为隐藏评分器的构建、编译或导入错误。修改与

核心差异。添加完成前验证门、SIGTERM/SIGALRM 始终终结和紧急终结、精确的接口/输出约束，以及更强的可靠性指导。

```
- success could finish without execution after an edit
+ pre-finish verification gate
+ always-finalize / emergency finalize
+ exact interface and output constraints
```

- 首先检查了思考块重放和提示缓存；延迟被拒绝作为主要杠杆。
- 假设：智能体自测并不意味着隐藏评分器接口的正确性；完成必须遵循真实执行。

SWE 73.0 Terminal 75.281 vs. H0: SWE +5, Terminal +1.12

02 为模型提供一次自我审查的真实 git 差异

T1 → T2 | 881b01b | 1 file | +68 / -0

观察与分析。在 T1 改进后，创建者检查了 728 处编辑，仅发现 14 个 str_replace_no_match 事件（1.8%），从而否定了更强编辑器的路线。其将剩余失败归因于多文件变更中的细微逻辑错误。修改与核心差异。添加 _current_diff() 和一次强制性的基于差异的审查。

self-review before a repository task ends.

```
- finish review relied on model memory
+ _current_diff()
+ mandatory diff-grounded self-review
```

- 该探针被明确视为一次带噪声的 n=5 崩溃检查。
- 该假设在变更前已写入台账。

SWE 75.0 Terminal 73.034 vs. T1: SWE +2, Terminal -2.25

03 将反思扩展到非 git 和终端任务

T2 → T3 | 76de0e6 | 1 file | +88 / -10 | selected final

观察与分析。许多终端工件是通过 shell 命令创建的，因此 H2 的 git 差异反思从未触发。创建者记录了 write_file/str_replace 路径和 run_command 状态，以便非 git 任务在完成前能够检查最近的工作。修改与核心差异。添加 _written_paths,

_ran_command, and _recent_workdir_files(); run artifact self-review for Terminal and non-git tasks.

```
- reflection only for repo tasks with git diff
+ written paths and command state
+ recent workdir files
+ non-git artifact self-review
```

- 所有 25 个 T2 SWE 失败均已触发反思。
- 第三轮检查发现了具体的接口不匹配问题，但仍侧重于一般性反思。

SWE 74.0 Terminal 74.157 vs. T2: SWE -1, Terminal +1.12

04 尝试终止进程组，随后撤销该操作

No formal T4 | 5faaa15 · LEDGER.md only | no full evaluation | session fork

观察与分析。创建者计算了跨版本并集与交集：软件工程任务中并集/交集/从未通过分别为 81/65/19，终端任务中为 71/59/18。反思仅在大约 4% 的情况下引发进一步修改。随后尝试使用 setsid 与 killpg 处理 Bash 超时。修改与核心差异。一个会话添加了

setsid/killpg，而另一个会话检出了 harness/tools.py 并撤销了修改。最终提交 5faaa15 仅更改了 LEDGER.md；不存在 H4 代码提交。

```
+ session A: setsid / killpg
- session B: revert tools.py
+ final commit changes LEDGER.md only
```

- 创建者认为，因一个边缘情况而改变 189 个任务的 Bash 行为风险过大。
- 两个并发会话对同一文件进行了相反的修改。

05 停止采样并选择第三轮

Final decision | declare-final 76de0e6 | 3 of 10 pairs used

观察与分析。创建者估计一次完整评估的噪声约为 $\pm 3-4$ 个任务，并判断另一个微不足道的提交只是额外采样，而非有意义的演化。它拒绝花费剩余七对样本去追逐更高的随机峰值。修改与核心差异。不添加候选者或

repeat samples; select T3 as the balanced, validated, mechanism-complete tie-break.

```
- more candidates or repeated samples
+ select T3 / 76de0e6
```

- 选定的 T3：软件工程 74.0，终端 74.157。
- T1 对高出约 0.06 个百分点，而 T1-T3 通过的任务总数相同。